arXiv:2601.20802v2 [cs.LG] 16 Feb 2026

Reinforcement Learning via Self-Distillation

Jonas Hübotter[1] **Frederike Lübeck**[*,1,2] **Lejs Behric**[*,1] **Anton Baumann**[*,1]
Marco Bagatella[1,2] **Daniel Marta**[1] **Ido Hakimi**[1] **Idan Shenfeld**[3]
Thomas Kleine Buening[1] **Carlos Guestrin**[4] **Andreas Krause**[1]
[1]ETH Zurich [2]Max Planck Institute for Intelligent Systems [3]MIT [4]Stanford

https://github.com/lasgroup/SDPO

Abstract

01.01 – Large language models are increasingly post-trained with reinforcement learning in verifiable domains such as code and math. Yet, current methods for reinforcement learning with verifiable rewards (RLVR) learn only from a scalar outcome reward per attempt, creating a severe credit-assignment bottleneck. Many verifiable environments actually provide rich textual feedback, such as runtime errors or judge evaluations, that explain *why* an
01.02 – attempt failed. We formalize this setting as reinforcement learning with rich feedback and introduce **Self-Distillation Policy Optimization** (**SDPO**), which converts tokenized feedback into a dense learning signal without any external teacher or explicit reward model. SDPO treats the current model conditioned on feedback as a self-teacher and distills its feedback-informed next-token predictions back into the policy. In this way, SDPO leverages the model's ability to retrospectively identify its own mistakes in-context.
01.03 – Across scientific reasoning, tool use, and competitive programming on LiveCodeBench v6, SDPO improves sample efficiency and final accuracy over strong RLVR baselines. Notably, SDPO also outperforms baselines in standard RLVR environments that only return scalar feedback by
01.04 – using successful rollouts as implicit feedback for failed attempts. Finally, applying SDPO to individual questions at test time accelerates discovery on difficult binary-reward tasks, achieving the same discovery probability as best-of-k sampling or multi-turn conversations with 3× fewer attempts.

1 Introduction

01.05 – Progress in deep reinforcement learning has shown that iterating on experience—acting, receiving feedback, and updating a policy—can unlock capabilities that are difficult to obtain from static supervision alone (Mnih et al., 2015; Silver et al., 2016; 2017; Berner et al., 2019). The same theme now appears in large language models (LLMs): large-scale post-training with reinforcement learning (RL) has substantially improved
01.06 – performance on reasoning-heavy tasks, especially in settings with programmatic or otherwise verifiable evaluation (Jaech et al., 2024; Guo et al., 2025; Kimi et al., 2025; Olmo et al., 2025).

Nevertheless, the dominant RL recipe for LLM post-training remains bottlenecked by

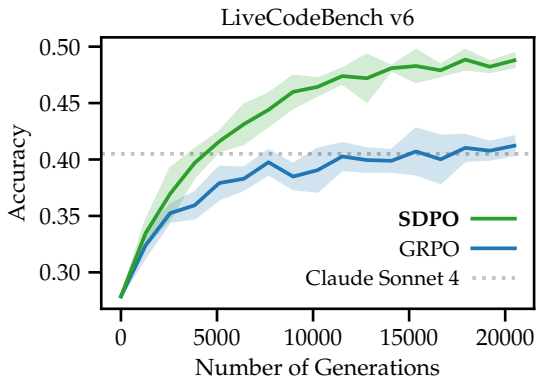


Figure 1: **SDPO substantially outperforms an improved version of Group Relative Policy Optimization (GRPO) on LCB v6 with Qwen3-8B.** Further, SDPO achieves GRPO's final accuracy in 4× fewer generations. Claude Sonnet 4 is the strongest instruct model on the public LCBv6 leaderboard. Shaded regions show the standard deviation across 3 seeds.

*Equal second authorship. Correspondence to jonas.huebotter@inf.ethz.ch.

01.01 段意

段意 | 研究问题：仅靠每次尝试的一个标量奖励，很难把成功或失败归因到具体生成位置。运行错误、评审意见等文本反馈提供了被传统 RLVR 丢弃的信息。

01.02 段意

段意 | 核心工作：提出 RLRF 设置及 SDPO 算法。当前模型看到反馈后担任教师，重新评估原回答的下一词元分布，再将此分布蒸馏回不看反馈的学生。

01.03 发现

发现 | 科学推理、工具使用和编程实验显示更好的准确率与样本效率；即使环境只有标量奖励，也可把同组成功答案当作失败尝试的反馈。

01.04 重点

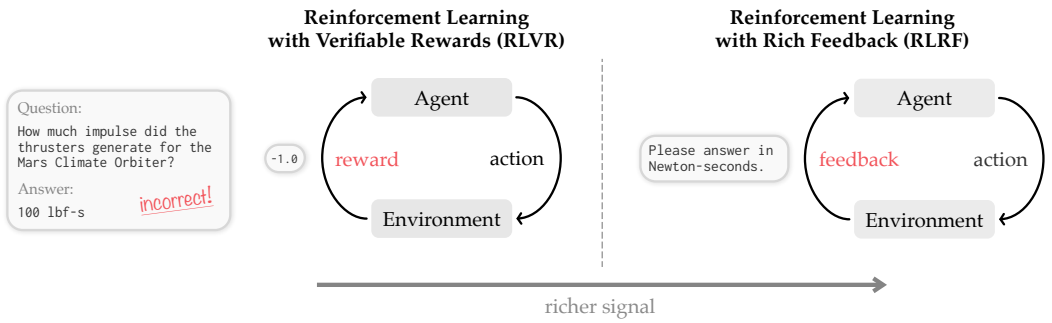
重点 | 单题测试时更新可提高难题的解题发现效率。摘要的“少 3 倍尝试”指达到特定发现概率时的比较，并非所有题或所有预算均快 3 倍。

01.05 段意

段意 | 引言背景：强化学习通过“行动—反馈—策略更新”积累经验，能获得静态监督不易带来的能力；本文研究 LLM 推理后训练中的这种机制。

01.06 图 1

图 1 | Qwen3-8B 的代码任务训练曲线：SDPO 最终准确率更高，达到 GRPO 最终准确率所需生成数约缩至四分之一。阴影表示 3 个种子的标准差。



02.01 – Figure 2: **Comparison of RLVR and RLRF settings.** In Reinforcement Learning with Verifiable Rewards (RLVR), the agent learns from a scalar reward r, which often acts as an information bottleneck by masking the underlying environment state. In contrast, Reinforcement Learning with Rich Feedback (RLRF) utilizes tokenized feedback. This provides a significantly richer signal than a scalar reward, as the feedback can encapsulate both the reward as well as detailed observations of the state (such as runtime errors from a code environment or feedback from an LLM judge).

02.02 – credit assignment. Most current approaches operate in the setting of reinforcement learning with verifiable rewards (RLVR): given a question x, the model samples an answer $y \sim \pi_\theta(\cdot \mid x)$ and receives a scalar reward $r \in \mathbb{R}$, often binary (e.g., unit-tests pass/fail in code generation). Modern policy gradient RLVR methods such as Group Relative Policy Optimization (GRPO; Shao et al., 2024) estimate advantages from these sparse outcome rewards. Furthermore, when all rollouts in a group receive the same (often zero) reward, GRPO advantages collapse to zero and learning stalls. To overcome this sparsity, one might prefer distillation from a strong teacher (Guo et al., 2025; Yang et al., 2025a; Lu & Thinking Machines Lab, 2025; Guha et al., 2026), which provides dense, token-level supervision. However, strong teachers are often unavailable in online learning, where the goal is to raise the capability ceiling beyond existing models.

02.03 – In this work, we argue that the key limitation is not RL per se, but the information bottleneck imposed by scalar outcome rewards. Many verifiable environments expose *rich tokenized feedback* beyond scalar rewards r, such as runtime errors, failing unit tests, or evaluations from an LLM judge. This feedback not only reveals *whether* a rollout was wrong, but also *what* went wrong. We formalize this more general setting as **Reinforcement Learning with Rich Feedback** (**RLRF**) and illustrate its difference to RLVR in Figure 2. Here, feedback can be any tokenized representation of any state reached by an agentic system. The central question becomes: how can we convert rich feedback into effective credit assignment without requiring external supervision from a strong teacher?

02.04 – Our starting point is the observation that LLMs already possess a powerful mechanism for using feedback: in-context learning (Brown et al., 2020; Wei et al., 2022). When conditioned on feedback, the same model can often identify plausible mistakes and propose a corrected approach. A common example of such feedback is the summary of failed test cases on coding platforms like LeetCode (Figure 3). Many recent works leverage this capability to iteratively generate corrections (Chen et al., 2021a; Madaan et al., 2023; Shinn et al., 2023; Yao et al., 2024; Yuksekgonul et al., 2025; Lee et al., 2025). In contrast, we use the current policy as a "self-teacher" that, rather than sampling a new response, re-evaluates the *existing* rollout after receiving rich feedback. Including the feedback in-context transforms the model's next-token distribution, allowing the self-teacher to agree or disagree with the student's original choices at specific tokens. This yields dense, logit-level credit assignment.

```
Runtime Error
ZeroDivisionError: division by zero
Line 73 in separateSquares (Solution.py)

Last Executed Input
[[26,30,2],[11,23,1]]
```

02.05 – Figure 3: Example of feedback from our code environment, inspired by LeetCode. Listings 4, 5, and 6 in the appendix show examples of feedback in case of a wrong answer, a memory error, and an index error.

02.01 图 2

图 2 | RLVR 仅使用奖励 r；RLRF 还使用可 token 化的环境反馈，例如指出单位错误或运行异常。反馈可同时编码成败和状态细节。

02.02 段意

段意 | 瓶颈：GRPO 根据同题一组回答的结果估计优势；若这一组奖励全相同，优势为零，无法继续提供学习方向。外部强教师虽能提供密集监督，但未必可用。

02.03 段意

段意 | 本文主张：问题出在标量奖励的信息压缩。丰富反馈不仅指出"错了"，也描述"哪里错"，研究目标是不用外部教师把这些信息变成信用分配。

02.04 段意

段意 | 机制起点：同一模型在看到反馈后，常能识别原回答的错误。SDPO 不再生成纠正版，而是重算原回答的条件概率，得到局部认可或否定信号。

02.05 图 3

图 3 | 除零异常、出错行与输入样例都是环境状态信息。它们本身不等于逐 token 奖励，需要教师结合原问题和生成前缀解释。

蓝色数字对应段落 / 图表；术语在首次出现处批注

Method	Sampling	Signal	Feedback
SFT / Distillation (Hinton et al., 2015)	× off-policy	✓ rich	× strong teacher
On-Policy Distillation (Agarwal et al., 2024)	✓ on-policy	✓ rich	× strong teacher
RLVR (such as GRPO) (Lambert et al., 2025)	✓ on-policy	× weak	✓ environment
RL via Self-Distillation (SDPO) (ours)	✓ on-policy	✓ rich	✓ environment

03.01 – Table 1: Comparison of self-distillation to alternative methods for post-training LLMs.

03.02 – For example, when provided with the feedback from Figure 3, the self-teacher can identify how the initial attempt should be modified to avoid the runtime error. Crucially, this mechanism incurs no sampling overhead: we simply re-compute the log-probabilities of the original attempt under the self-teacher's feedback-augmented context.

03.03 – Building on this idea, we introduce **Self-Distillation Policy Optimization** (**SDPO**), an on-policy algorithm that performs RL via self-distillation. SDPO samples rollouts from the current policy, obtains rich environment feedback, and then minimizes a logit-level distillation loss that matches the current policy's next-token distribution to that of the self-teacher. Conceptually, SDPO addresses the central limitation of applying distillation to online learning: the absence of a stronger external teacher. Instead of relying on a fixed teacher, SDPO leverages the model's ability to recognize its own mistakes in hindsight. By conditioning the current policy on the rich feedback it just received, we construct a self-teacher that provides the dense supervision of distillation while retaining the exploration benefits of on-policy RL. Table 1 summarizes how this positions SDPO relative to RLVR and distillation baselines. We include a comprehensive summary of related work in Section 6.

03.04 – We show that SDPO is a policy gradient algorithm whose advantages are estimated using the self-teacher. This enables the implementation of SDPO with minor changes to standard RLVR pipelines, simply by swapping out the advantages.

Summary of evaluation results. We evaluate SDPO in three online RL settings:

03.05 –
- **Learning without rich feedback** (§3): We evaluate standard RLVR environments that do not return any feedback beyond scalar rewards. Here, SDPO treats successful attempts sampled in the current batch as "feedback" for failed attempts on the same question. We perform training runs on scientific reasoning and tool use, starting with Qwen3-8B and Olmo3-7B-Instruct. We find that SDPO outperforms a strong GRPO baseline that integrates recent improvements: 70.2% vs. 66.6% final accuracy on aggregate. SDPO achieves higher accuracy with up to 11× shorter generation lengths compared to GRPO, demonstrating that effective reasoning need not be verbose.

03.06 –
- **Learning with rich feedback** (§4): We evaluate competitive programming problems from LiveCodeBench v6 with LeetCode-style feedback. As shown in Figure 1, SDPO substantially improves over GRPO, reaching a higher final accuracy (48.8% vs. 41.2%) and achieving GRPO's final accuracy in 4× fewer generations. SDPO's gains grow with model scale, suggesting that the ability for self-teaching emerges as models become stronger in-context learners.

03.07 –
- **Discovering novel solutions to hard tasks at test-time** (§5): Finally, we demonstrate that SDPO can accelerate the discovery of solutions to difficult binary-reward questions. This contrasts with RLVR methods, which only begin learning once the first solution has been found. We leverage SDPO for **Test-Time Self-Distillation**, a form of test-time training where the model specializes to an individual test question. We consider very difficult LiveCodeBench questions, for which the base model's pass@64 is below 0.03, and show that SDPO accelerates the discovery of solutions by 3×.

2 SDPO: Self-Distillation Policy Optimization

03.08 – We propose an algorithm that uses the in-context learning ability of the current policy for assigning credit. Our key object is the *self-teacher*, $\pi_\theta(\cdot \mid x, f)$, which refers to the

03.01 表 1

表 1 | 方法定位：传统蒸馏常依赖强教师；RLVR 在线采样但反馈稀疏；SDPO 用环境反馈和自身模型，结合在线探索与分布级监督。

03.02 段意

段意 | 无需教师再采样：只在反馈增强的上下文下重新计算原尝试的对数概率。因此“不增加生成次数”不代表“没有额外前向计算”。

03.03 段意

段意 | 算法概述：从当前策略采样、获取反馈、构造自我教师，再匹配学生与教师的下一词元分布。额外信息来自环境，而非预先存在的更强模型。

03.04 重点

重点 | SDPO 梯度可写成策略梯度形式，优势由自我教师估计。这使已有 RLVR 训练管线可通过替换优势项来实现。

03.05 实验 1

实验 1 | 标量环境中，以当前组内成功回答为反馈。总体准确率 70.2% 对 66.6%，部分设置回答长度缩短至 GRPO 的约 1/11；并非所有任务都达到该幅度。

03.06 实验 2

实验 2 | LCBv6 代码任务：最终准确率 48.8% 对 41.2%；模型越强，相对收益通常越明显，提示自我回顾依赖上下文学习能力。

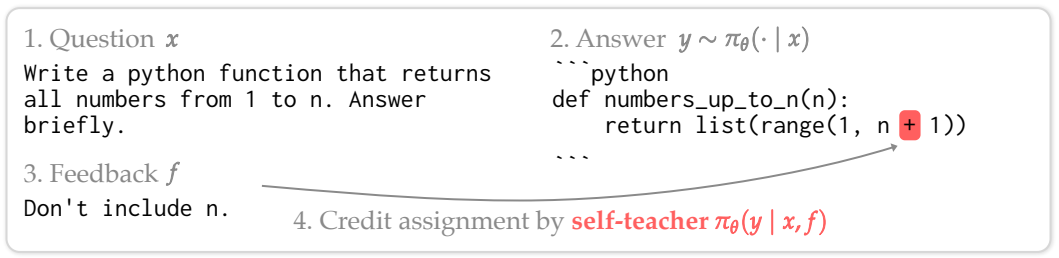
03.07 实验 3

实验 3 | 针对单个难题做测试时自蒸馏。即使尚未找到正确解，也可利用运行反馈更新策略；后文用 discovery@k 衡量这种搜索过程。

03.08 段意

段意 | 方法对象：学生和教师共享模型来源，差别在于教师额外看到反馈。教师并非天然更强，而是具有额外的事后信息。

蓝色数字对应段落 / 图表；术语在首次出现处批注



04.01 – Figure 4: Example of self-teaching with Qwen3-8B. The answer is generated by the model before seeing the feedback. Then, we re-evaluate the log-probs of the original attempt with the *self-teacher* after seeing the feedback. We show the per-token $\log(\mathbb{P}(\text{self-teacher})/\mathbb{P}(\text{student}))$, with red indicating negative values (self-teacher disagrees) and white indicating values around zero. Notably, in this example, Qwen3-8B identifies the error through retrospection without an explicit solution. Further, the activation is sparse, identifying where mistakes happen and adjusting to the students' response distribution.

04.02 – current policy (the "student") prompted with the question x and the rich feedback f. Next to the students' original attempt y, f may incorporate two key kinds of feedback: any environment output (such as runtime errors from a code environment) and a sample solution if x was already solved with another attempt in the rollout group.[1] As discussed before, the self-teacher $\pi_\theta(\cdot \mid x, f)$ should have a higher accuracy than the student $\pi_\theta(\cdot \mid x)$ since it sees additional information in-context. This leads us to observe:

04.03 – **We can use the same policy in two different roles: As the student for the initial attempt and as the teacher to determine the value of actions in hindsight.**

04.04 – We introduce **Self-Distillation Policy Optimization** (**SDPO**) which repeatedly distills the self-teacher into the student. Given a question x, we first sample rollouts from the student π_θ and obtain corresponding environment feedback. We then use the KL-divergence, $\mathrm{KL}(p\|q) = \sum_i p(i) \log p(i)/q(i)$, as a distance measure for the next-token distributions of student and teacher, and optimize a standard logit distillation loss:

$$\mathcal{L}_{\text{SDPO}}(\theta) := \sum_t \mathrm{KL}(\pi_\theta(\cdot \mid x, y_{<t}) \| \text{stopgrad}(\pi_\theta(\cdot \mid x, f, y_{<t}))) \tag{1}$$

04.05 – where the stopgrad operator blocks gradients from flowing through the teacher, and thus prevents it from regressing towards the student and ignoring f. The intuitive role of the teacher is to determine where and how the students' original attempt y was wrong through retrospection based on the feedback f. Figure 4 shows an example of self-teaching with Qwen3-8B as student and self-teacher. We summarize SDPO in Algorithm 1 and display the teachers' reprompt template in Table 2.

Algorithm 1 SDPO

Input: Language model π_θ; dataset with questions x; number of rollouts G per question; environment to obtain feedback for attempts.

1: **repeat**
2: Sample question x from dataset.
3: Sample responses: $\{y_i\}_{i=1}^{G} \sim \pi_\theta(\cdot \mid x)$.
4: Evaluate responses to obtain feedback f_i.
▷ **Self-distillation:**
5: Compute log-probs of self-teacher $\log \pi_\theta(y_{i,t} \mid x, f_i, y_{i,<t})$.
6: Update θ with gradient descent on $\mathcal{L}_{\text{SDPO}}(\theta)$.
7: **until** converged

We can derive the SDPO gradient as follows (see Appendix A.1 for details):

04.06 – **Proposition 2.1.** *The gradient of* $\mathcal{L}_{\text{SDPO}}$ *is*

$$\nabla \mathcal{L}_{\text{SDPO}}(\theta) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)}\left[\sum_{t=1}^{|y|} \mathbb{E}_{\hat{y}_t \sim \pi_\theta(\cdot|x, y_{<t})}\left[\log \frac{\pi_\theta(\hat{y}_t \mid x, y_{<t})}{\pi_\theta(\hat{y}_t \mid x, f, y_{<t})} \cdot \nabla_\theta \log \pi_\theta(\hat{y}_t \mid x, y_{<t})\right]\right]. \tag{2}$$

[1] In standard RLVR implementations a rollout group contains multiple simultaneous attempts for x.

04.01 图 4

图 4 | 原回答包含 n，反馈要求不包含 n。教师在相关代码词元处降低原选择的概率，说明无需显式正确解，也可能定位局部错误。

04.02 段意

段意 | 教师输入：问题、生成前缀及反馈；反馈可含环境输出和组内成功样例。额外信息通常有助于回顾，但并不保证教师每次判断都正确。

04.03 重点

重点 | 同一策略承担两个角色：学生先尝试，教师事后根据反馈判断动作。改进来自信息条件差异，而非额外引入专家模型。

04.04 工作

工作 | 式 (1) 对每个已采样前缀最小化学生到教师的 KL。这里匹配的是下一词元分布，不只是模仿一条纠正版答案。

04.05 段意

段意 | 停止教师梯度，避免优化时教师向学生退让并忽略反馈。算法 1 的循环是：采样、环境评估、教师重算、学生梯度更新。

04.06 公式精读

公式精读 | 令 p 为学生、q 为停止梯度的教师，A=log(q/p)，则 □L=−E[A□log p]。梯度下降会提高 q/p 较大的候选词元概率，压低较小者。

User:	prompt Correct solution: successful_previous_rollout The following is feedback from your unsuccessful earlier attempt: environment_output Correctly solve the original question.
Assistant:	original_response

05.01 – Table 2: Template for self-teacher. prompt is replaced with the question. A sample solution previously generated by the student is substituted for successful_previous_rollout (if available for this question; otherwise the paragraph is skipped). environment_output is replaced with the environment output (see, e.g., Figure 3) from the models' original attempt (if it was not successful and there is no solution; otherwise the paragraph is skipped). If the models' original attempt was successful, this attempt is passed as the correct solution. original_response is replaced with the models' original attempt to re-evaluate its log-probabilities under the self-teacher.

2.1 Comparison to RLVR

05.02 – Note that the SDPO gradient is a (negated) logit-level policy gradient where the advantages are estimated using the self-teacher.[2] We can therefore reuse standard RLVR implementations and simply swap out the advantages. Let y_i be the i-th rollout from a rollout group of size G for question x, then comparing GRPO and SDPO we have:

$$A_{i,t}^{\text{GRPO}} := r_i - \text{mean}\{r_i\}_{i=1}^{G} \text{ (constant in } t\text{)}, \quad A_{i,t}^{\text{SDPO}}(\hat{y}_{i,t}) = \log \frac{\pi_\theta(\hat{y}_{i,t} \mid x, f_i, y_{i,<t})}{\pi_\theta(\hat{y}_{i,t} \mid x, y_{i,<t})}.$$

05.03 – The GRPO advantages are applied only to the sampled token $y_{i,t}$ and are constant within a rollout y_i.[3] In contrast, the SDPO advantages are zero only for tokens where student and teacher perfectly agree. The SDPO advantage is positive for tokens which are more likely under the teacher while being negative for tokens which are less likely under the teacher. Thus, SDPO can be seen as a direct extension of standard RLVR methods in two ways:

1. from 1-bit feedback to *allowing arbitrary sequences of tokens as feedback*, and
2. leveraging this rich feedback to *estimate dense logit-level advantages*.

05.04 – This tight connection to RLVR methods also enables a straightforward extension of the SDPO gradient from Equation (2) to off-policy data via PPO-style clipped importance sampling (Schulman et al., 2017), see Appendix A.4.

2.2 Compute time & memory

05.05 – The only computational overhead of SDPO compared to GRPO is the additional computation of log-probs from the self-teacher, which can be effectively parallelized and is substantially faster than sequential generation. Figure 5 compares the compute time of SDPO and GRPO. As expected, the compute overhead of SDPO is relatively small. Here, we use a micro batch size of 2;[4] compute time can be further reduced by using larger micro batch sizes.

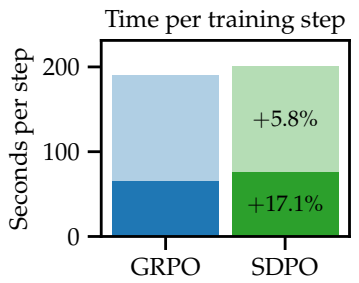


Figure 5: Time per step for SDPO vs GRPO (solid: without code environment, light: with code environment).

05.06 – Naively computing the KL divergence between student and teacher requires holding full logits of both models in memory. To avoid this, we approximate the KL divergence

[2] See Appendix A.4 for a detailed comparison of the SDPO gradient to the standard policy gradient.
[3] We use the GRPO (Shao et al., 2024) advantage without normalization (Liu et al., 2025b).
[4] The micro batch size corresponds to # rollouts we train on at a time while accumulating gradients.

05.01 模板

模板 | 成功样例优先来自同题组内学生回答；没有成功解时可使用失败反馈。末尾放回原始 response，用教师对它评分，而非要求教师另采样一条训练答案。

05.02 段意

段意 | 与 RLVR 的连接：SDPO 是带分布级优势的负策略梯度损失。工程上可复用标准策略优化代码，但须保留优势的逐位置、逐候选词元结构。

05.03 对比

对比 | GRPO 的优势在同一回答中对位置 t 恒定，只更新采样词元；SDPO 由教师与学生分布是否一致决定，可区分该位置的多个候选词元。

05.04 扩展

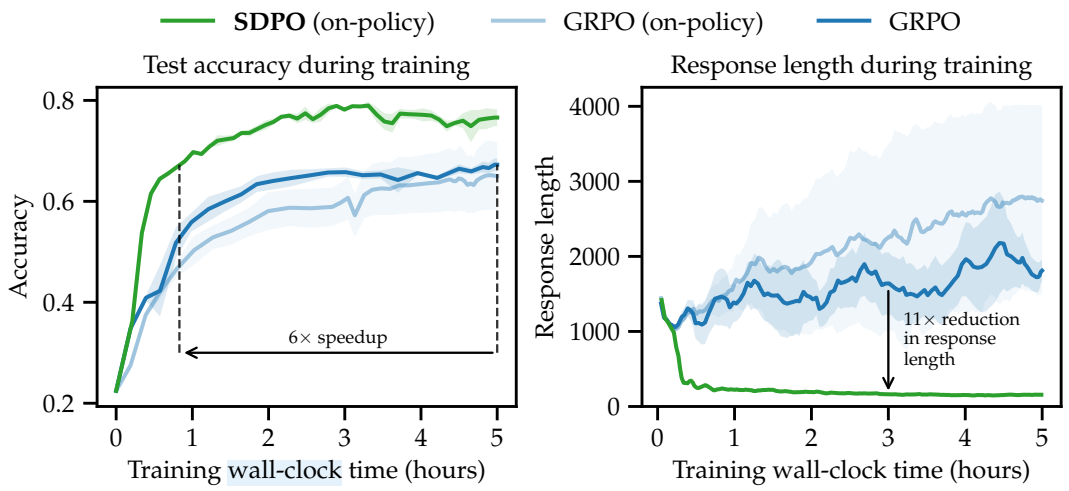
扩展 | 离策略数据可结合 PPO 式裁剪重要性采样校正。附录 A.4 给出分布级推广，不能直接把所有词元都当成相同的序列权重。

05.05 计算

计算 | 主要额外开销是教师前向重算，易于并行，通常比自回归生成快。图 5 的开销依是否计入代码环境时间而不同。

05.06 显存

显存 | 完整 KL 需要保存两套大词表 logits；作者采用学生 top-K 候选加尾部质量的近似，具体公式见附录 A.3。



06.01 – Figure 6: Training progression of Olmo3-7B-Instruct on Chemistry. We report the average accuracy across 16 samples per question and a rolling average of response lengths over 5 steps. We report GRPO with the optimal hyperparameters for this model and task. We run each configuration for 3 seeds and report standard errors as shaded areas.

06.02 – in the SDPO loss by performing top-K distillation (i.e., only computing the top-K logits of the student and the corresponding logits of the teacher alongside a term capturing the tail probability; cf. Appendix A.3). With a reasonable choice of K (e.g., $K = 100$), this avoids virtually any memory overhead while capturing most of the information.

2.3 Stability improvements

06.03 – We find that two practical modifications significantly enhance the training stability of SDPO. First, we employ a regularized self-teacher, implemented either via an exponential moving average (EMA) of the student parameters or by interpolating the current teacher with the initial teacher (cf. Appendix A.2). As detailed later, both strategies effectively stabilize learning. Second, we adopt the symmetric Jensen-Shannon divergence for the distillation loss; this formulation has similarly been shown to improve stability in on-policy distillation from external teachers (Agarwal et al., 2024).

3 Learning without Rich Environment Feedback

06.04 – We first evaluate SDPO in standard RLVR environments, where feedback is limited to scalar rewards. Instead of using the scalar reward, SDPO treats successful attempts sampled in the current batch as "feedback" for failed attempts on the same question. By comparing the student's attempt with a correct solution, the self-teacher can identify where the student was wrong and provide dense credit assignment.

3.1 Experimental setting

06.05 – We evaluate tasks on which the model has not been explicitly fine-tuned:

- **Science Q&A** (Chemistry, Physics, Biology, Materials science): Undergraduate-level scientific reasoning using reasoning subsets (L3) from SciKnowEval (Feng et al., 2024a).
- **Tool use**: Mapping a tool-API specification and user request to the correct tool call, using ToolAlpaca (Tang et al., 2023).

06.06 – We perform a train-test split to test in-domain generalization. We use Qwen3-8B (Yang et al., 2025a) and Olmo3-7B-Instruct (Olmo et al., 2025) as initial checkpoints and report avg@16 relative to wall-clock training time, excluding initialization & validation.

06.01 图 6

图 6 | Olmo3 化学任务中，SDPO 更快提升准确率并明显缩短回答；“6 倍时间加速”和“11 倍长度缩减”属于这一具体设置。

06.02 段意

段意 | top-K 近似：保留学生最可能的 K 个词元及对应教师概率，剩余质量合并为 tail，兼顾显存和分布信息。

06.03 稳定性

稳定性 | 两种改动：用 EMA 或参考教师插值正则化教师；采用 JS 散度。具体实验未全部使用同一种散度，应结合表 12 查看。

06.04 段意

段意 | 无丰富环境反馈时，用组内成功回答指导失败回答。此时成功样例由学生自身产生，不要求专家解，但全组失败时仍缺少这种反馈来源。

06.05 实验设置

实验设置 | 任务含大学科学推理与工具调用；采用训练/测试划分，检验域内泛化，基模为 Qwen3-8B 和 Olmo3-7B-Instruct。

06.06 指标

指标 | avg@16 是每题 16 次采样的平均正确率，不是“至少一次正确”的 pass@16。时间曲线排除了初始化和验证耗时。

	Chemistry		Physics		Biology		Materials		Tool use	
	1h	5h	1h	5h	1h	5h	1h	5h	1h	5h
Qwen3-8B	41.2		59.2		30.8		58.9		57.5	
+ GRPO	65.9	74.5	63.8	72.7	35.1	**59.9**	**74.3**	77.1	64.9	67.7
+ GRPO (on-policy)	63.3	63.4	63.6	63.6	49.8	49.8	73.9	74.1	60.2	65.7
+ **SDPO** (on-policy)	**73.2**	**80.9**	**66.6**	**75.6**	**50.6**	56.8	72.1	**78.4**	**68.0**	**68.5**
Olmo3-7B-Instruct	22.8		37.7		16.2		36.7		39.3	
+ GRPO	39.7	56.7	55.3	63.3	35.6	**55.8**	70.9	75.0	56.4	**65.0**
+ GRPO (on-policy)	51.4	57.5	**62.7**	62.7	**49.8**	49.8	73.3	73.5	56.8	60.6
+ **SDPO** (on-policy)	**68.0**	**80.0**	59.9	**66.1**	48.0	52.8	**73.7**	**79.1**	**60.8**	62.1

07.01 – Table 3: **Comparison of SDPO and GRPO on reasoning-related benchmarks.** We report the highest achieved avg@16 within 1 hour and 5 hours of wall-clock training time, respectively. Both SDPO and on-policy GRPO perform one gradient step per generation batch, while GRPO performs 4 off-policy mini batch steps. We select optimal hyperparameters for SDPO
07.02 – and baselines based on 5h accuracy. Each run is performed on a node with 4 NVIDIA GH200 GPUs. Together with initialization and validation, each run takes approximately 6 hours.

Baselines. We compare SDPO to an improved variant of **GRPO** (Shao et al., 2024), which incorporates several recent modifications (Olmo et al., 2025; Khatri et al., 2026) such as asymmetric clipping (Yu et al., 2025), avoiding biased normalization (Liu et al., 2025b), and correcting for off-policy data when using efficient inference frameworks (Yao et al., 2025). We integrate these modifications into a GRPO implementation that represents a strong baseline, as detailed in Equation (12) in Appendix A.4. GRPO enables off-policy training through PPO's clipped importance weighting (Schulman et al., 2017). We additionally report the special case of **on-policy GRPO** (matching the hyperparameters of vanilla SDPO). For both baselines, we perform a hyperparameter sweep and report results for the models that achieve the highest validation performance across all target tasks. Hyperparameters and training details are provided in Appendix E. We use the `verl` library (Sheng et al., 2025) for fast multi-GPU training.

3.2 Results

07.03 – Table 3 summarizes our results. We find that SDPO outperforms GRPO across almost all runs, often leading to substantial improvements. SDPO learns notably faster than GRPO, performing close to 5 hours of GRPO training after only 1 hour of training with SDPO in several cases. SDPO achieves a particularly substantial improvement over GRPO on the Chemistry task, as is displayed in Figure 6 (left). With Olmo3-7B-Instruct, *SDPO achieves the 5h GRPO accuracy in 50 minutes of wall-clock training time*, a 6× speedup. Moreover, SDPO's 5h accuracy is more than 10%-points higher than that of GRPO.

07.04 – We remark that our results with SDPO use strictly on-policy training (i.e., one gradient step per generation batch). Given the known efficiency gains of off-policy methods that perform multiple gradient updates per generation batch, we believe that studying SDPO with off-policy updates is an exciting direction for future work.

Takeaway 1

07.05 – We demonstrate that SDPO can learn to reason effectively, generalizing to challenging reasoning tasks. Without requiring any modification to existing RLVR environments, SDPO outperforms GRPO substantially in several cases.

3.3 Self-distillation learns to reason concisely

07.06 – We consistently observe that SDPO produces substantially shorter generations than GRPO while achieving higher accuracy. SDPO's responses are more than 3× shorter on average

07.01 表 3

表 3 | 报告 1h/5h 内达到的最高 avg@16。SDPO 和 on-policy GRPO 每批一次更新，常规 GRPO 使用 4 次离策略小批更新；要注意优化预算差异。

07.02 基线

基线 | GRPO 加入非对称裁剪、去除有偏归一化和推理策略差异校正，并对各方法调参；本文比较对象是改进版 GRPO。

07.03 结果

结果 | 多数任务 SDPO 学得更快；Olmo3 化学任务约 50 分钟达到 5 小时 GRPO 的准确率，最终还高出十余个百分点。生物等任务存在例外。

07.04 边界

边界 | 主实验 SDPO 严格每个生成批次只更新一次。作者把充分探索多次离策略更新视为后续方向，不能据此认定离策略一定无益。

07.05 小结

小结 | 即使不改变 RLVR 环境，利用成功样例做自蒸馏也可在多项任务上增强推理；结论是多数设置受益，并非每个任务均胜出。

07.06 发现

发现 | SDPO 在保持较高准确率时往往回答更短；下一页将区分有效推理与重复、循环的表面推理。

蓝色数字对应段落 / 图表；术语在首次出现处批注

…Alternatively…
Closer to D? No…
Wait I'm going in circles… Wait, perhaps the correct answer is B… $10^{1.85} \approx 69.3$…
Ah, this works… Wait I think I messed up…
Hmm… $10^{1.85} \approx 69.3$…
Thus, the correct answer is likely B: 1.85.
<answer>
B
</answer>

(a) GRPO (5,549 tokens)

…At pH 7.4, all functional groups are neutral… maintaining a balance between hydrophobic and hydrophilic character… [The] overall polarity… keeps logD from being very high… or very low… [typically falling] in the 2.0-3.0 range, with 2.61 (C) being a reasonable estimate…
<answer>
C
</answer>

(b) SDPO (764 tokens)

08.01 – Figure 7: Example responses from GRPO and SDPO after 50 training steps to the following question: "What is the correct octanol/water distribution coefficient logD under the circumstance of pH 7.4 for the molecule `O=C1O[C@@H](COc2ccon2)CN1c1ccc(C2=CCOCC2)c(F)c1`?" The answer options are A: 1.32, B: 1.85, C: 2.61, D: 3.76. The correct answer is **C**. GRPO's answer contains 5× "Hmm.", 9× "No.", and 25× "Wait". Further, GRPO's answer repeats calculations such as "$10^{1.85} \approx 69.3$", which appears four times, and the model even explicitly generates "Wait I'm going in circles". SDPO's answer avoids any circular reasoning and is more than 7× shorter. The base model is Qwen3-8B.

08.02 – across tasks (cf. Table 8 in Appendix D). On Chemistry with Olmo3-7B-Instruct, SDPO even achieves an 11× reduction in response length relative to GRPO while maintaining higher accuracy (Figure 6 (right)). While recent progress in RLVR has demonstrated that scaling response length is a powerful driver of emergent reasoning capabilities (Jaech et al., 2024; Guo et al., 2025; Muennighoff et al., 2025), our results suggest that effective reasoning need not always be verbose. We find that SDPO improves the *efficiency* of reasoning.

08.03 – Qualitatively, we observe that the longer responses from GRPO often stem from "superficial" reasoning rather than necessary analytical steps. GRPO frequently generates filler phrases like "Hmm" and "Wait" or enters circular logical loops that repeat previous steps verbatim. Figure 7 displays a representative example of this phenomenon. Remarkably, SDPO's generations remain concise and avoid these superficial patterns. This may be explained by SDPO's dense credit assignment, which assigns a specific advantage to each next-token prediction, leading to sparse advantages (cf. Figure 21 in Appendix F). By improving the efficiency of reasoning, SDPO reduces inference generation time and demonstrates that reasoning performance can be improved by refining *how* the model reasons, not just how *long* it reasons.

4 Learning with Rich Environment Feedback

08.04 – We next evaluate SDPO on coding tasks. Coding is a canonical example of an RL environment that provides rich feedback, such as runtime errors and failed unit tests. Learning to solve these coding problems requires strong credit assignment since the student must identify its precise mistakes to avoid repeating them in the future. LiveCodeBench (LCB; Jain et al., 2025) provides a set of contest-style coding problems, ranging from simple to competition-level. We restrict our evaluation to the most recent LCBv6 subset of LCB, which contains 131 questions released between February and May 2025. We consider a setting with public and private unit tests, common for code contests and coding platforms like LeetCode, where the public tests are used for evaluation during training and the private tests are used for validation (Chen et al., 2022; Le et al., 2022; El-Kishky et al., 2025; Samadi et al., 2025).[5]

08.05 – We use the Qwen3 (Yang et al., 2025a) model family for our experiments, with Qwen3-8B as default unless otherwise specified. We report the average accuracy over 4 rollouts and use the same GRPO baseline as outlined in Section 3.1.

[5]We select public tests as a 50% random subset of private tests.

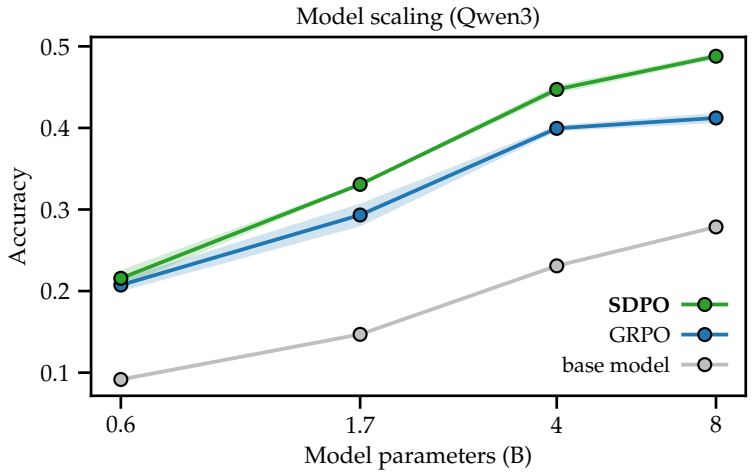
08.01 图 7
图 7 | GRPO 样例有重复计算和多次自我否定，SDPO 更短且答对。它是定性例子，整体趋势仍需结合跨任务长度统计。

08.02 段意
段意 | 平均生成长度约缩短三倍，特定化学设置缩短十一倍。更长链式推理虽常有效，本文提示也可通过提高推理效率来改进。

08.03 解释
解释 | 作者推测分布级信用分配抑制冗余片段。这是机制解释，不能仅凭长度下降推断每段被删内容都无用。

08.04 编程实验
编程实验 | LCBv6 含 131 道 2025 年 2—5 月题目。训练使用公开测试反馈，验证使用私有测试；脚注说明公开测试由私有测试随机抽取 50%。

08.05 评测
评测 | 默认 Qwen3-8B，用 4 次 rollout 的平均准确率。这里主要检验同一题目对未用于反馈的测试用例泛化，并非完全未见题目的泛化。



09.01 – Figure 8: **SDPO improves with model size.** We compare the final LCBv6 validation accuracy of SDPO and GRPO at train step 80, across model sizes from Qwen3. The ability of SDPO's teacher to perform accurate retrospection appears to be an emergent phenomenon with scale. We include an additional scaling study with Qwen2.5-Instruct in the appendix (cf. Figure 17) which further supports this finding. Error bars indicate the standard error across 3 seeds.

09.02 – **Results.** Figure 1 compares the learning curves of SDPO and GRPO on LCBv6. We find that SDPO achieves a substantially higher final accuracy (48.8%) than GRPO (41.2%) while also outperforming the strongest instruct models on the public LCBv6 leaderboard:[6] Claude Sonnet 4 (40.5%) and Claude Opus 4 (39.7%). Furthermore, SDPO reaches the final accuracy of GRPO in 4× fewer generations. We include an extended comparison to other RLVR baselines that perform similarly to GRPO in Table 9 in the appendix. Differentiating between the easy, medium, and hard questions of LCB, we find that SDPO particularly improves over GRPO in solving medium and hard questions (cf. Figure 15 in the appendix).

4.1 Self-distillation benefits from stronger models

09.03 – A central question for our work is whether SDPO is sensitive to the in-context learning ability of the base model. Intuitively, we expect that SDPO benefits from a strong in-context learner, since this enables the teacher to perform more accurate retrospection.

09.04 – To answer this question, we perform a scaling study with different model sizes from the Qwen3 (Yang et al., 2025a) family. As shown by extensive prior work, the ability to learn in-context increases with model size (e.g., Brown et al., 2020). As depicted in Figure 8, SDPO significantly outperforms GRPO on larger models while only slightly improving over GRPO on smaller models. To determine whether SDPO can also underperform GRPO on a model weaker than Qwen3-0.6B, we performed an additional scaling study with Qwen2.5-Instruct (Qwen et al., 2024). While outperforming GRPO with Qwen2.5-7B and performing similarly with Qwen2.5-8B, we find that SDPO underperforms GRPO on Qwen2.5-1.5B, as seen in Figure 17 in Appendix D.

Takeaway 2

09.05 – Our results suggest that the marginal improvement of SDPO over GRPO is tightly coupled with the strength of the base model, and motivates future study on models stronger than Qwen3-8B. In the same way that in-context learning is an emergent phenomenon with scale, the self-teacher's ability to perform accurate retrospection in SDPO appears to be emergent with scale.

[6] On the public leaderboard, the LCBv6 subset can be obtained by selecting February to May 2025.

09.01 图 8

图 8 | 在统一训练步附近比较 Qwen3 不同规模。SDPO 优势随规模变大更明显，支持“回顾能力依赖基模”的解释。

09.02 结果

结果 | SDPO 最终验证准确率 48.8%，GRPO 41.2%；达到 GRPO 最终水平所需生成数约为其四分之一。与公开榜单模型对比时需留意评测协议是否一致。

09.03 研究问题

研究问题 | 教师来自基模的上下文学习能力，因此强基模是否更能利用反馈，是方法适用范围的关键。

09.04 规模消融

规模消融 | 大 Qwen3 收益更明显；Qwen2.5 的最小模型可能落后 GRPO。跨系列除参数量外还受预训练和指令微调影响。

09.05 小结

小结 | 作者把自我回顾视为随规模增强的能力。当前证据来自有限模型系列，扩展到更强基模仍是待研究方向。

4.2 Self-distillation performs dense credit assignment

10.01 – Whereas GRPO assigns a constant advantage to each generated token, SDPO assigns an individual advantage to *each possible next token* along the generated sequence based on the agreement of student and teacher. At each position t in the generated sequence y, there are $|\mathcal{V}|$ possible next tokens where $\mathcal{V}$ is the vocabulary. In distillation, this level is typically called the *logit-level* since it corresponds to the logits of the model. In practice, we approximate the full next-token distribution by the top-K tokens plus the tail, and as such, SDPO assigns $|y| \cdot (K+1)$ unique advantages per sequence. This is illustrated in Figure 9 and allows SDPO to perform dense credit assignment.

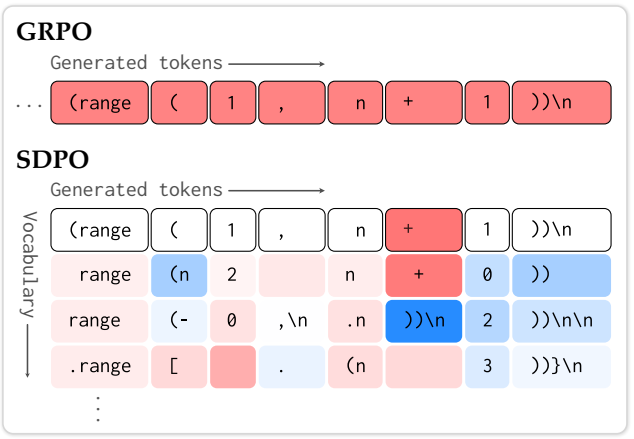


Figure 9: Dense credit assignment in SDPO in the example from Figure 4. Shown in blue are tokens which become more likely under the self-teacher. The self-teacher identifies how the returned range has to be modified so that it does not contain n.

10.02 – A natural question is whether the performance gains of SDPO are due to leveraging rich feedback in RLRF or due to the dense credit assignment of SDPO. To answer this question, we ablate the performance of SDPO in three configurations:

10.03 –
- **Logit-level SDPO:** credit assignment over the 100 most likely tokens (under the student) at each position.
- **Token-level SDPO:** credit assignment over the most likely token at each position.
- **Sequence-level SDPO:** We compute SDPO advantages for all generated tokens and average them to produce a single scalar advantage per sequence (as in GRPO). This does not perform denser credit assignment than GRPO but still leverages the rich feedback f.

10.04 – As shown in Figure 10 (left), the dense credit assignment of logit-level SDPO leads to significant performance gains over token-level SDPO and sequence-level SDPO. Nevertheless, even sequence-level SDPO outperforms GRPO, indicating that leveraging rich feedback in RLRF can lead to substantial gains over RLVR methods even without dense credit assignment.

4.3 The self-teacher improves during training

10.05 – Contrary to standard distillation, the self-teacher in SDPO is not frozen, but updated throughout training. This is a critical component of SDPO, since it enables the teacher to improve over time, which means that the student can learn from a stronger target. To investigate whether the self-teacher improves during training, we plot the average accuracy when *generating* using the self-teacher in Figure 10 (right). We find that the self-teacher improves significantly during training. Most notably, the student's accuracy surpasses the initial teacher's accuracy in later stages of training. This demonstrates that SDPO enables true bootstrapping of a weak model to a strong model, without the initial self-teacher's performance limiting the final student.

Teacher	Accuracy	Avg accuracy
q_θ	36.1 ± 1.6	29.8 ± 1.3
$q_{\theta_{\text{ref}}}$	48.8 ± 0.7	44.4 ± 0.2
Trust-region	$\mathbf{50.6} \pm 0.9$	$\mathbf{45.6} \pm 0.2$
EMA	49.3 ± 0.3	$\mathbf{45.3} \pm 0.2$

Table 4: Best/average accuracy until step 90 of various methods for teacher regularization. Trust-region and EMA teachers use $\alpha = 0.01$. Training of the q_θ eventually diverges. Error ranges indicate standard errors across 3 seeds.

10.06 –

中文精读 · Wang | 原文第 10 页

10.01 段意

段意 | logit-level 优势覆盖每个位置的多个候选词元；采用 top-K 加尾项后，每条长度 T 的序列有约 T×(K+1) 个优势，远细于一个序列分数。

10.02 消融目的

消融目的 | 把“获得丰富反馈”和“细粒度分配信用”分开检验，避免将所有收益都归结为其中一个因素。

10.03 消融设置

消融设置 | 分布级保留 top-100 候选；词元级只保留一个候选；序列级把生成词元优势平均成一个标量。三者逐渐丢弃位置信息与候选信息。

10.04 发现

发现 | 分布级优于词元级和序列级；但序列级 SDPO 仍超过 GRPO。丰富反馈与更细的信用分配分别贡献收益。

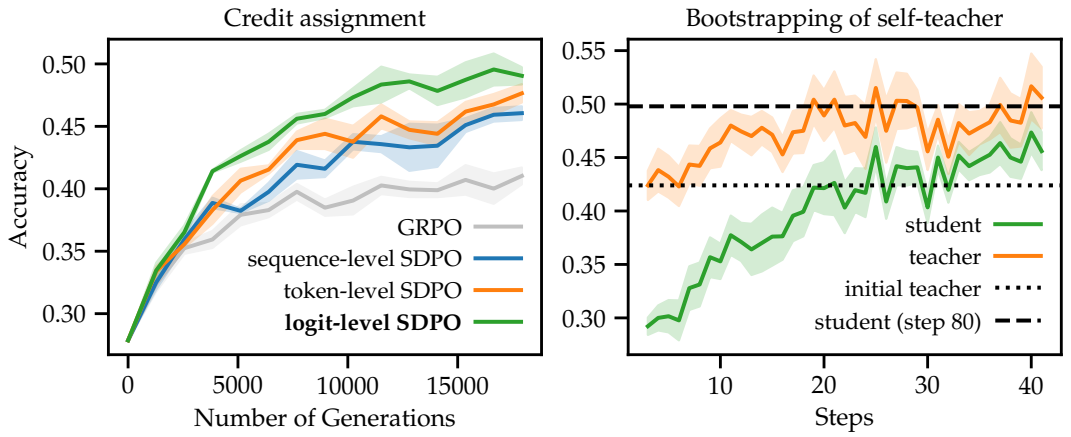
10.05 自举

自举 | 教师随训练更新，学生后期甚至超过初始教师，说明初始教师准确率并非硬上限；教师共享更新也可能造成不稳定。

10.06 表 4

表 4 | 无正则教师较差且可发散；EMA 与信赖域教师优于冻结教师。要区分固定教师基线与不断改进的正则化教师。

蓝色数字对应段落 / 图表；术语在首次出现处批注



11.01 – Figure 10: **Left: Rich feedback in RLRF and dense credit assignment of SDPO are complementary.** We compare logit-level, token-level, and sequence-level SDPO advantages to GRPO. While denser credit assignment in SDPO is beneficial (logit-level > token-level > sequence-level), even sequence-level SDPO significantly outperforms GRPO due to leveraging the rich feedback. Error bars indicate the standard error across 3 seeds. **Right: The self-teacher improves during training.** We display the generative accuracy of the self-teacher compared to student on the current training batch (with a rolling average over 5 steps). The final student score is taken at step 80. Notably, the performance of the student significantly surpasses the initial teacher's accuracy. Error bars indicate the standard deviation across 3 seeds.

11.02 – As described in Section 2.3, SDPO uses a regularized teacher to stabilize training. As can be seen in Table 4, a non-regularized teacher significantly underperforms the regularized teachers. Furthermore, trust-region and EMA teachers outperform the teacher frozen at the initial teacher's parameters, showing that the teacher improves through parameter sharing with the student. Yet, SDPO performs well even with a frozen teacher.

4.4 On-policy self-distillation avoids catastrophic forgetting

11.03 – Prior work has shown that a key benefit of on-policy algorithms, such as GRPO, is that models tend not to forget previously obtained capabilities (Shenfeld et al., 2026b; Chen et al., 2025b; Lu & Thinking Machines Lab, 2025). This is practically desirable since it enables continual training pipelines where a model is trained sequentially on diverse tasks without the need to retrain from scratch. To evaluate forgetting, we test the final checkpoints of GRPO and SDPO on diverse holdout tasks: IFEval (Zhou et al., 2023), which tests the ability of a model to follow precise format instructions; ArenaHard-v2 (Li et al., 2025a), which is an LLM-judged benchmark of real-world instruction-following prompts derived from LMArena (Chiang et al., 2024); and MMLU-Pro (Wang et al., 2024b), which tests broad multi-task knowledge and reasoning. As displayed in Table 5, SDPO learns the new task while mitigating degradation of initial capabilities, overall achieving a better performance–forgetting tradeoff than GRPO.

11.04 – **Off-policy self-distillation baseline.** As an additional baseline, we consider training the student via supervised fine-tuning (SFT) on successful generations from the self-teacher (Scheurer et al., 2023; Dou et al., 2024; Zhou et al., 2025).[7] This requires 2× the generations of SDPO for the same number of steps, since we have to generate from both the student and the teacher. We report SFT on the successes of the self-teacher, which achieves a higher accuracy than also including initial successes from the student in the SFT data. As shown in Table 5, SFT on the self-teacher significantly underperforms SDPO on LCBv6, while leading to worse forgetting of prior capabilities. This mirrors prior findings on the instability of off-policy imitation (see, e.g., Agarwal et al., 2024).

[7] SFT on a teacher's predictions is a standard off-policy distillation approach (Kim & Rush, 2016).

11.01 图 10

图 10 | 左侧比较优势粒度，右侧比较学生与当前/初始教师。右侧教师在训练批次上的曲线与最终学生分数并非完全相同的采样条件。

11.02 段意

段意 | 教师正则化稳定学习；EMA 和信赖域版本较冻结教师更强，支持教师随学生改善的自举作用，冻结教师也仍能产生收益。

11.03 保留能力

保留能力 | 用 IFEval、ArenaHard-v2 和 MMLU-Pro 检查跨任务遗忘。作者关注学习代码任务后是否损伤原有指令遵循与知识能力。

11.04 SFT 对照

SFT 对照 | 用教师成功生成做监督微调需要额外教师采样；它在代码准确率和能力保留上逊于 SDPO，说明模仿教师答案与修正自身轨迹并不等价。

蓝色数字对应段落 / 图表；术语在首次出现处批注

	Task:	Holdout tasks:				
	LCBv6	IFEval	ArenaHard-v2 (hard prompt)	ArenaHard-v2 (creative writing)	MMLU-Pro	**Avg.** (holdout)
Base	27.9	83.9	14.0	13.7	62.5	43.5
SFT on self-teacher	42.7	83.7	11.2	8.9	61.9	41.4
GRPO	41.2	82.2	12.0	10.8	62.3	41.8
SDPO	48.8	83.2	12.3	11.1	62.9	42.4

12.01 – Table 5: **On-policy methods do not suffer from catastrophic forgetting.** We compare the accuracy of the final checkpoint on the training task LCBv6 and on holdout tasks IFEval, ArenaHard-v2, and MMLU-Pro. We compare to a baseline that trains directly on responses generated by the initial self-teacher with SFT. Overall, SDPO achieves the best performance–forgetting tradeoff. We include additional baseline results in Table 9 in the appendix.

4.5 Can GRPO and SDPO be combined?

12.02 – GRPO utilizes Monte Carlo advantages, which are unbiased with respect to the objective of maximizing expected reward $J(\theta) := \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[r(y \mid x)]$. In contrast, SDPO advantages are inherently biased with respect to $J(\theta)$ due to being computed from rich feedback and a self-teacher. This dichotomy parallels the fundamental distinction between Monte Carlo and bootstrapped advantages in RL: while the latter are biased, they typically yield lower variance (Sutton & Barto, 1998; Schulman et al., 2016). This motivates a hybrid approach that combines reward-derived GRPO advantages with feedback-derived SDPO advantages:

$$A_{i,t}^{\text{SDPO+GRPO}}(\hat{y}_{i,t}) := \lambda A_{i,t}^{\text{GRPO}}(\hat{y}_{i,t}) + (1-\lambda)A_{i,t}^{\text{SDPO}}(\hat{y}_{i,t}), \quad \lambda \in [0,1]. \tag{3}$$

12.03 – As shown in Figure 11, SDPO+GRPO appears to be more robust to weaker models than SDPO. Intuitively, in a weaker model such as Qwen3-0.6B, the SDPO advantages are less reliable, and hence including the GRPO advantage helps to stabilize training. In contrast, we find that SDPO+GRPO slightly underperforms SDPO on stronger models such as Qwen3-8B. This suggests that the signal of GRPO, only informed by a scalar reward, can be actively harmful with a strong initial model.

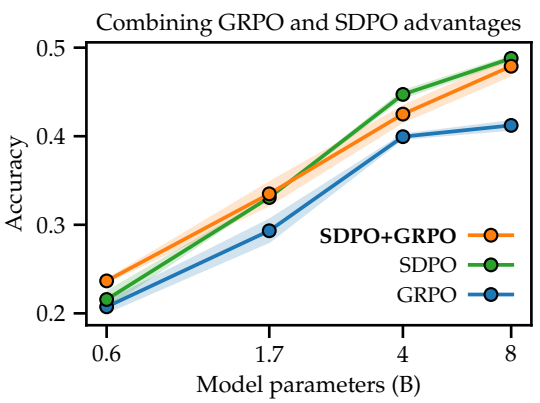


Figure 11: We compare the LCBv6 validation accuracy at step 80, across model sizes from Qwen3. SDPO+GRPO significantly outperforms SDPO on the weaker Qwen3-0.6B, while slightly underperforming SDPO on stronger models. We use $\lambda = 0.9$. Error bars indicate the standard error across 3 seeds.

4.6 Which feedback is most informative?

12.04 – To understand which type of rich feedback is most informative, we ablate the three types of feedback present in a verifiable environment like code generation: the sample solution (if a successful rollout is available in the current rollout group), the environment output (such as runtime errors), and the student's original attempt.

12.05 – **Sample solutions.** Including a sample solution from a failed attempt's rollout group (if available) closely mirrors the group-relative advantages of GRPO. We emphasize that these sample solutions are always generated by the student, as in GRPO, and do not require an expert model. They allow for disincentivizing unsuccessful approaches if the model is already able to solve the question. However, unlike GRPO where all tokens receive the same negative advantage, the self-teacher can identify specific mistakes and provide feedback on how to fix them.

12.01 表 5

表 5 | SDPO 代码准确率最高，留出任务平均分也高于两种训练基线；但仍略低于基模。因此结论应是缓解遗忘，而非完全没有遗忘。

12.02 偏差—方差

偏差—方差 | GRPO 的蒙特卡洛优势针对期望奖励构建；SDPO 优势依赖教师和反馈，对该奖励目标一般有偏。自举信号可有较低方差，却不自动保证方向正确。

12.03 混合结果

混合结果 | 式 (3) 线性组合两种优势。较弱模型可由 GRPO 信号稳定训练；强模型中加入标量优势略降表现，可能稀释密集教师信息。

12.04 反馈研究

反馈研究 | 比较环境输出、同组成功解和原始尝试三个来源，分别检验它们对教师信息量和后续学生训练的影响。

12.05 成功解

成功解 | 来源是学生在同题当前组内的成功 rollout，无需专家。它提供可行替代路径，使教师能定位失败答案的具体错误，而非一律压低所有词元。

	Teacher before training		Student trained with SDPO	
	↑ Acc. (%)	↓ Same output (%)	↑ Acc. (%)	Avg. entropy
f = output	32.5 ± 0.5	13.7 ± 0.6	39.9 ± 1.1	0.40 ± 0.0
f = own solution	$\mathbf{42.4} \pm 1.0$	12.1 ± 0.7	42.6 ± 1.3	0.41 ± 0.0
f = output + own solution	$\mathbf{42.5} \pm 1.2$	$\mathbf{10.1} \pm 0.2$	$\mathbf{48.3} \pm 1.4$	0.38 ± 0.0
$f = y$ + output + own solution	39.3 ± 0.8	30.0 ± 0.9	44.5 ± 1.3	$\mathit{0.23} \pm 0.0$

13.01 – Table 6: **Performance of varying kinds of feedback.** We evaluate informativeness of feedback based on SDPO training (until step 60) as well as the direct impact on the self-teacher. 13.02 – "Same output" measures the percentage of cases where the teacher receives the same environment output as the student's initial attempt (i.e., not exploring alternative approaches). We observe that environment output and sample solutions are complementary and each provide informative feedback. Naively including only solutions or initial attempts y significantly reduces diversity in the teacher and student. We remark that the sample solutions are generated by the student, enabling similar group-relative advantage estimation to GRPO. Error bars indicate standard deviation across 3 seeds.

Environment output. The environment output describes the state of the environment after the student's attempt. This is complementary to sample solutions since it can provide useful signal even if the student has never solved the question before (a setting we explore extensively in Section 5). Leveraging environment output is a key differentiating factor between RLRF and RLVR settings.

13.03 – **Student's original attempt.** The student's original attempt y does not have to be included in the reprompting template of the teacher. Indeed, we find that including it biases the teacher towards the student's attempt (cf. Table 6). This reduces the entropy of the student's distribution (particularly for initially uncertain tokens), thereby reducing exploration.

13.04 – We summarize results in Table 6 where we evaluate the effect on SDPO training as well as the direct impact on the self-teacher. We find that environment output & sample solutions are complementary, each providing informative feedback. Generally, we observe that performance is not sensitive to syntactic variations of the reprompting template from Table 2.

5 Solving Hard Questions via Test-Time Self-Distillation

13.05 – In Sections 3 and 4, we have demonstrated that SDPO can substantially improve over RLVR methods when performing "train-time RL" for reasoning tasks. We now turn to a test-time setting where the model is given only a single hard (binary-reward) question x and must discover a solution as quickly as possible:

13.06 – **Definition 5.1** (Discovery time)**.** *The discovery time is the number of trials needed until a solution is found (i.e., the smallest k with the k-th attempt y_k receiving reward 1).*

Based on this notion, we can define a measure of the efficacy of discovery:

$$\begin{aligned} \text{discovery@}k &:= \mathbb{P}(\text{discovery time} \leq k) \\ &= \mathbb{P}(r(y_1 \mid x) = 1 \text{ or } r(y_2 \mid x) = 1 \text{ or } \ldots \text{or } r(y_k \mid x) = 1), \end{aligned} \tag{4}$$

where the probability is over any randomness in the algorithm producing y_k and the rewards. Thus, the discovery@k metric quantifies the probability of discovering the solution within k steps.[8] While prior work has studied discovery with continuous rewards (e.g., Novikov et al., 2025; Yuksekgonul et al., 2026), discovery with language models in sparse or binary-reward settings does not allow "hill-climbing" a continuous reward and has remained less well understood.

13.07 – The most naive approach to discovery in binary-reward tasks is to sample repeatedly i.i.d. from the base model, also known as **best-of-k**. The canonical pass@k metric for

[8] Our proposed discovery@k metric is a canonical metric in the study of runtime speedup (i.e., time until termination, Dolan & Moré (2002)).

13.01 表 6

表 6 | 环境输出与自身成功解结合最好；额外放入原尝试时，教师更容易重复原环境输出，学生熵下降。教师直接生成的初始准确率并不能完全预测训练收益。

13.02 段意

段意 | 运行反馈反映失败后的状态，即使从未答对也可有信息；成功样例则说明可行解法，两类来源互补。

13.03 段意

段意 | 把原答案额外放入教师提示可能诱发确认偏差、降低探索；注意教师重算响应概率所用的前缀，与提示里额外重复整条回答不是一回事。

13.04 发现

发现 | 两种反馈结合效果好，模板的简单措辞变化影响较小；"模板鲁棒"不意味着可以忽略反馈来源与内容结构。

13.05 新设置

新设置 | 从跨题训练转向单个难题的测试时适应：尽快找到第一条正确答案，模型权重可在尝试过程中更新。

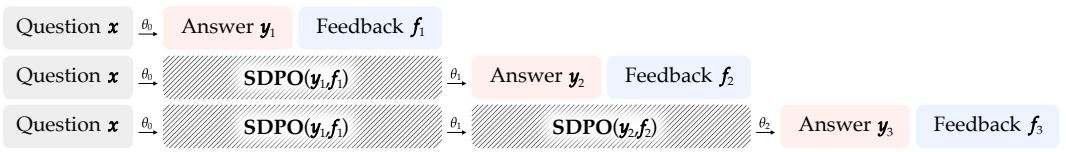
13.06 定义

定义 | discovery time 是首次成功所需尝试数；discovery@k 是 k 次以内至少成功一次的概率，可用于比较会动态更新的搜索算法。

13.07 基线

基线 | best-of-k 对固定模型独立重复采样，不利用之前尝试的信息；后页与保留上下文的多轮采样、更新权重的 SDPO 对比。

蓝色数字对应段落 / 图表；术语在首次出现处批注



14.01 – Figure 12: **Compressing context into model weights via self-distillation.** We illustrate the process of distilling the interaction history (context c) into the model parameters θ. The model π_θ repeatedly attempts a fixed hard question x, generating an answer y and receiving feedback f. Rather than appending this history to the context window, the model updates its weights $\theta_t \to \theta_{t+1}$ with SDPO (batch size 1) based on the feedback, effectively "fixing" mistakes by encoding $\pi_\theta(\cdot \mid x, c)$ directly into the policy $\pi_{\theta'}(\cdot \mid x)$.

14.02 – best-of-k sampling is exactly the probability of discovering at least one solution within k independent samples from a fixed model, coinciding with discovery@k. The discovery@k metric generalizes pass@k to algorithms that sample attempts sequentially. A common sequential approach re-prompts the base model with additional context from previous attempts (Madaan et al., 2023; Shinn et al., 2023). We refer to this as **multi-turn** sampling. Here, the model itself does not change, only its context evolves over time.

14.03 – Performing RLVR on the question x does not improve over best-of-k sampling from the base model, since a binary reward provides no signal until the first solution has already been found.[9] An RLRF method like SDPO does not face the same limitation, as it receives rich feedback from the environment after each attempt. This rich feedback enables the model to repeatedly "correct" its mistakes as it encounters them and receives feedback, even before ever discovering a solution. In contrast to multi-turn sampling, SDPO repeatedly compresses context $c = (y_k, f_k)$ by distilling $\pi_\theta(\cdot \mid x, c)$ into a model $\pi_{\theta'}(\cdot \mid x)$ as we illustrate in Figure 12. This self-distillation enables SDPO to continually learn over long contexts, whereas the memory bottleneck of transformers inherently limits the context length of multi-turn sampling (Vaswani et al., 2017). In this section, we seek to answer the question:

Can repeatedly compressing context into model weights via self-distillation accelerate discovery for hard questions?

5.1 Experimental setting

14.04 – We consider a particularly challenging subset of questions from LCBv6 that are at Qwen3-8B's performance ceiling and require significant test-time sampling to find any solution. Concretely, we define two groups using Qwen3-8B's pass@k: *Hard tasks* with pass@64 < 0.5 and *very hard tasks* with pass@64 < 0.03. Among these, we retain questions for which any of best-of-k, multi-turn, or SDPO find at least one solution within 512 steps across 5 seeds. This results in 19 hard and 9 very hard questions.

14.05 – For best-of-k sampling under the base model, we report the standard pass@k estimate (Chen et al., 2021b) from 2944 independent rollouts. As multi-turn sampling, we sequentially reprompt the model in-context using the concatenated feedback from previous attempts. To remain within Qwen3-8B's 40k-token context limit, we employ a first-in, first-out sliding window, discarding the earliest feedback once the maximum prompt length (32k tokens) is reached. We ablate the multi-turn reprompting strategy in Figure 19 in Appendix D and find that retaining only past feedback while forgetting earlier attempts significantly outperforms the baseline that additionally retains past attempts. We evaluate SDPO with a batch size of 16. We ablate this choice in Figure 19 in Appendix D and find that overall performance differences are marginal, yet smaller batch sizes are beneficial for improvements at low generation budgets, while larger batch sizes result in more stable updates that still learn to solve questions at later stages into the run.

[9] For this reason, several works consider explicitly constructing curricula of solvable questions (e.g., Zhao et al., 2025; Huang et al., 2026; Diaz-Bone et al., 2025; Hübotter et al., 2025b), which self-distillation avoids. Other work found that RLVR yields limited improvement on hard questions (Yue et al., 2025).

14.01 图 12

图 12 | 每轮把回答及环境反馈中的经验蒸馏进权重；无需把完整历史一直留在提示中。这是把上下文信息转成参数更新的示意。

14.02 指标联系

指标联系 | 固定模型独立采样时，discovery@k 与 pass@k 一致；多轮采样或权重更新时，它还能处理前后尝试不独立的情况。

14.03 重点

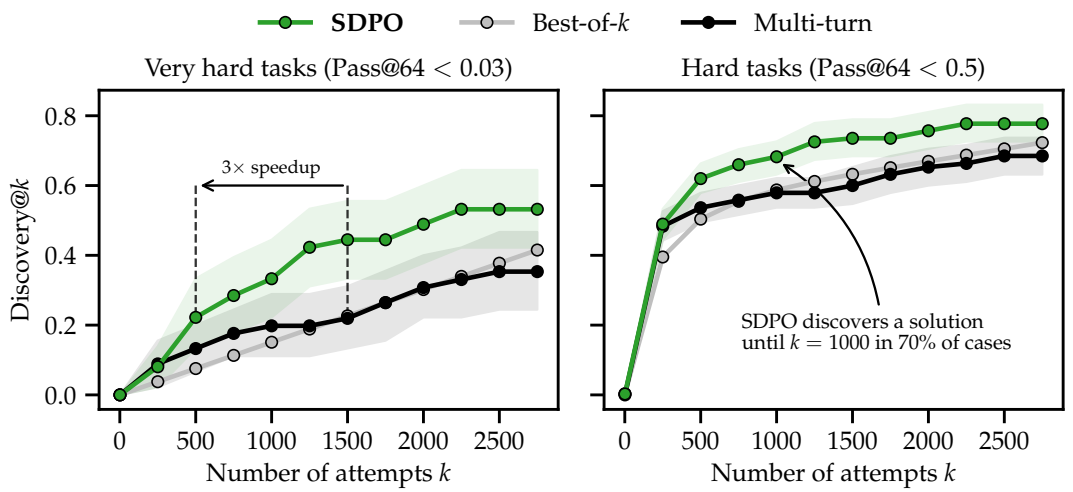
重点 | 在本文二元组相对奖励设置下，首次成功之前没有奖励差异；丰富失败反馈可让 SDPO 在此之前学习。多轮对话只改变上下文，SDPO 还更新参数。

14.04 筛选条件

筛选条件 | 困难题 pass@64<0.5，极难题 <0.03；还保留至少一种方法在筛选预算内找到解的题目，最终 19/9 道。存在筛选效应，不能外推为所有难题的成功率。

14.05 比较协议

比较协议 | best-of-k 用 2944 条独立 rollout 估计；多轮基线用滑动窗口保留反馈；SDPO 每批 16 条。尝试数衡量生成预算，未计入梯度更新的全部计算成本。



15.01 – Figure 13: **Self-distillation at test-time solves LiveCodeBench questions that neither the base model nor multi-turn conversations can solve. Left:** Very hard questions (9 total) from LCBv6 where the base model achieves pass@64 < 0.03, i.e., in less than 3% cases, sampling 64 responses yields any success. **Right:** Hard questions (19 total) from LCBv6 where the base model achieves pass@64 < 0.5. We report the discovery@k metric, representing the probability of discovering at least one solution within k total generations. Across both difficulty levels, SDPO achieves higher discovery@k rates at almost all generation budgets, compared to the base model and a multi-turn conversation baseline that receives the feedback in-context. We report the mean and bootstrapped 90% confidence intervals of the mean across 5 random seeds per question.

5.2 Results

15.02 – 15.03 – Figure 13 compares discovery@k for SDPO, multi-turn sampling, and best-of-k sampling on very hard (left) and hard (right) questions from LCBv6. Across both difficulty levels, SDPO achieves substantially higher discovery@k rates at almost all generation budgets.

On very hard tasks, multi-turn and best-of-k largely fail to solve questions within the available generation budget, achieving discovery@2750 of only 35.6% and 41.5%, respectively, whereas SDPO discovers a solution in 53.2% of cases. SDPO not only solves more questions overall but also does so with substantially fewer attempts. Notably, to reach a 22% discovery probability on very hard questions, SDPO requires approximately 3× fewer generations than best-of-k and multi-turn sampling. On hard tasks, SDPO reaches a 78% discovery@2750 probability while achieving a 67% discovery probability with roughly 2.4× fewer generations than best-of-k and multi-turn sampling. Overall, multi-turn and best-of-k sampling solve only 68.4% and 72.3% of questions, respectively. The context window length for multi-turn sampling is reached after 837 (±466) steps for hard questions and after 1007 (±349) steps for very hard questions, offering a possible explanation for its diminishing gains at high generation budgets.

15.04 – **Question 3 is only solved by SDPO.** SDPO solves all questions that are solved by best-of-k and multi-turn sampling. Beyond that, SDPO uniquely discovers a solution for Q3, which is neither solvable with multi-turn sampling nor with best-of-k sampling within 2750 attempts. In contrast, SDPO first discovers a solution for Q3 after 321 attempts, which corresponds to 20 iteration steps of self-distillation based on feedback with a batch size of 16. We include detailed per-question results in Table 10 in Appendix D.

15.05 – **The initial self-teacher does not solve hard questions.** Notably, the self-teacher's initial accuracy is $< 1\%$ for almost all questions, and even exactly 0% on 78% of them (Table 11 in Appendix D). This shows that a single turn of in-context feedback is insufficient to solve the

15.01 图 13

图 13 | 报告各题 5 个随机种子的均值及 bootstrap 90% 置信区间；极难题仅 9 道，困难题 19 道，样本量限制结论精度。

15.02 结果概述

结果概述 | 多数生成预算下，SDPO 的至少一次成功概率较高；这里是搜索发现概率，而非单次输出的平均准确率。

15.03 数字精读

数字精读 | 极难题 2750 次预算：SDPO 53.2%，best-of-k 41.5%，多轮 35.6%。约 3 倍加速对应达到 22% 发现概率；困难题达到 67% 时约 2.4 倍。

15.04 个案

个案 | Q3 在基线预算内未解决，SDPO 某次首次在第 321 次尝试成功；该数不是跨种子的平均首次成功次数，附录表 10 给出另一种汇总。

15.05 发现

发现 | 多数难题初始教师成功率几乎为零，但局部信用信号仍可支持逐步改进；“直接解不出”不等于“无法指出任何局部错误”。

problem. Despite this, the self-teacher's credit assignment is sufficiently effective for SDPO to iteratively refine the policy and eventually solve these questions.

Takeaway 3

16.01 – We demonstrate that rich environment feedback enables SDPO to significantly accelerate discovery for hard questions. This is in contrast to RLVR methods, which only receive a binary reward signal, and therefore only begin learning once the first solution has already been found.

6 Related Work

6.1 Reinforcement Learning with LLMs

16.02 – Recently, large-scale RL training on diverse tasks has significantly improved the performance of LLMs on general reasoning tasks (Guo et al., 2025; Kimi et al., 2025; Olmo et al., 2025; Jaech et al., 2024; Lambert et al., 2025). This progress is primarily enabled by RLVR methods that use Monte Carlo estimates of rewards, such as STaR or GRPO (Zelikman et al., 2022; Shao et al., 2024), similar to the classical REINFORCE algorithm (Williams, 1992). While several traditional RLVR algorithms rely on learning separate value networks (Schulman et al., 2017), they incur substantial memory costs and retain the information bottleneck of scalar rewards.

16.03 – In the RLVR setting, it is common for an (outcome) reward to be given only at the end of a sequence. To improve credit assignment, several works learn so-called process reward models (PRMs) that estimate rewards for each step in the sequence (Lightman et al., 2023; Wang et al., 2024a; Setlur et al., 2025). Unlike our RLRF setting, PRMs are typically trained on scalar rewards, either on value estimates for intermediate states or on outcome rewards (Cui et al., 2025). Unlike the self-teacher in SDPO, PRMs are a distinct model from the student, introducing significant memory overhead. Our work shows that *each language model is implicitly a PRM* through retrospection if given rich feedback.

16.04 – Conceptually, our work is related to "bootstrapping your own latent" (BYOL; Grill et al., 2020) and "expert iteration" (Anthony et al., 2017) where a student is bootstrapped by repeatedly imitating an improved version of itself (called the "expert"). Canonically, the expert combines the student with test-time search, such as tree search (Anthony et al., 2017) or majority voting (Zuo et al., 2025). In contrast, SDPO leverages the student's ability to learn from rich feedback provided in-context, which is related to "augmented views" in BYOL.

6.2 Learning from Rich Feedback and through Retrospection

16.05 – Beyond scalar outcome rewards, recent works have leveraged rich execution or verbal feedback to guide generation (Gehring et al., 2025; Feng et al., 2024b; Yuksekgonul et al., 2025). A primary line of research focuses on translating verbal feedback into reward functions for RL. This is often achieved by mapping feedback to discrete token-level rewards using an external frozen model (Wang et al., 2026), or by employing strong external LLMs to explicitly construct state-wise reward functions (Goyal et al., 2019; Xie et al., 2024; Urcelay et al., 2026).

16.06 – Alternatively, feedback can be utilized without explicit reward modeling. Several approaches focus on in-context improvement without integrating the process into the RL optimization loop (Chen et al., 2021a; Madaan et al., 2023; Shinn et al., 2023; Yao et al., 2024; Yuksekgonul et al., 2025; Lee et al., 2025). Others manually curate preference datasets by pairing responses before and after feedback to train with direct preference optimization (Stephan et al., 2024; Lee et al., 2024), though this requires additional generation and lacks the direct credit assignment of SDPO. Various recent works bootstrap thinking traces from known answers, using these answers as rich feedback (Zhou et al., 2026; Hatamizadeh et al., 2026; Zhang et al., 2025).

16.07 – A central object in several recent works is a feedback-conditioned policy $\pi_\theta(y \mid x, f)$, which learns answers y that lead to feedback f (Liu et al., 2023; Zhang et al., 2023; Luo et al., 2025),

16.01 小结

小结 | 丰富环境反馈允许在首次成功前更新。本文对纯二元 RLVR 的限制描述应结合组相对优势和任务设定理解。

16.02 相关工作

相关工作 | 传统 RLVR 常用采样回报估计优势；价值网络可使位置级估计更细，但增加存储/训练成本，也仍可能受标量奖励信息量限制。

16.03 PRM 对比

PRM 对比 | 过程奖励模型估计中间步骤好坏，常是独立训练的模型；SDPO 通过同一模型看反馈后的回顾形成隐式过程评估，省去独立奖励网络。

16.04 概念联系

概念联系 | 自举与 expert iteration 不断模仿更强的自身版本。SDPO 的"更强"来自反馈增强的视角，而非树搜索构造的专家。

16.05 路径一

路径一 | 既有工作可用外部模型把自然语言反馈转成标量或逐步奖励；SDPO 不需显式进行这种奖励函数构造。

16.06 路径二

路径二 | 反思式生成与偏好数据整理也能用反馈，但往往需额外生成，或不在在线优化环中直接做原轨迹的细粒度信用分配。

16.07 区别

区别 | 反馈条件策略常把反馈作为目标条件；本文把反馈当作事后状态证据，用来评价实际尝试中的动作。

17.01 – typically through supervised objectives. The idea behind these approaches is to deploy a policy conditioned on desirable (i.e., positive) feedback for deployment. This approach is conceptually related to goal-conditioned RL (Schaul et al., 2015; Liu et al., 2025a), where one can learn from negative examples through goal relabeling (Andrychowicz et al., 2017). Feedback-conditioned policies view feedback as a goal, whereas RLRF views feedback as a state that can be used to determine whether the goal x is achieved. Unlike SDPO, these methods do not use feedback for credit assignment in negative trajectories, but rather as a data transformation for goal relabeling.

6.3 Distillation

17.02 – Distillation is frequently employed as an alternative to supervised fine-tuning (SFT) when a strong teacher model is available. Distillation transfers capabilities by training a student to mimic the output distribution or intermediate representations of the teacher (Hinton et al., 2015; Romero et al., 2015; Kim & Rush, 2016; Sanh et al., 2019; Xie et al., 2020). While often performed on fixed off-policy datasets, to address the distribution shift between training and inference, recent works explore on-policy distillation, where the student learns from feedback on its own generations provided by an external teacher (Agarwal et al., 2024; Gu et al., 2024; Yang et al., 2025a; Lu & Thinking Machines Lab, 2025). This mitigates the train-test mismatch, which relates closely to earlier work on online imitation learning (Ross et al., 2011).

6.4 Self-Distillation

17.03 – The concept of self-distillation was first proposed by Snell et al. (2022) in a setting akin to supervised learning, introducing the idea of sampling from a model provided with extra context and training the same model to mimic these predictions without that context. This mechanism has proven effective for compressing behavior (Bai et al., 2022; Choi et al., 2022; Yang et al., 2024; 2025b) and factual information (Eyuboglu et al., 2026; Kujanpää et al., 2025; Cao et al., 2025a) into model weights. Beyond compressing a fixed context into model weights, recent works have used self-distillation to learn from environment feedback (Scheurer et al., 2023; Dou et al., 2024; Zhou et al., 2025; Mitra & Ulukus, 2025; Song et al., 2026). These approaches use an *off-policy* self-distillation objective, which we find to substantially underperform SDPO's on-policy learning. Off-policy self-distillation trains the student on generations from the teacher, whereas SDPO trains the student to avoid mistakes in its own generations.

17.04 – In concurrent work, Chen et al. (2025c) apply on-policy self-distillation to grid world settings where feedback is a scalar reward, and a reflection stage in the self-teacher diagnoses possible mistakes, showing improved credit assignment compared to learning value networks for advantage estimation. Other concurrent work studies SDPO on a fixed dataset of expert demonstrations, without online environment interaction (Shenfeld et al., 2026a; Zhao et al., 2026).

7 Conclusion, Limitations, and Future Work

17.05 – We introduced **Reinforcement Learning with Rich Feedback** (RLRF), a paradigm where environments provide tokenized feedback beyond scalar rewards, and argued that this removes a key information bottleneck of RLVR. We then proposed **Self-Distillation Policy Optimization** (SDPO), which uses the current policy as a feedback-conditioned *self-teacher* and distills its corrected log-probabilities into the student. This leverages the model's ability to learn from context for dense credit assignment. We further demonstrated that SDPO can be implemented as a minimal, drop-in modification to standard RLVR pipelines.

17.06 – Empirically, SDPO demonstrates superior sample efficiency and wall-clock convergence compared to GRPO on reasoning tasks, even when training in standard RLVR environments without rich feedback. SDPO's gains grow with model scale, suggesting that the capacity for self-correction scales with the model's in-context learning capabilities. Moreover, we show that performing SDPO at test time on individual hard binary-reward tasks accelerates the discovery of solutions compared to strong baselines.

17.07 – SDPO enables learning from rich feedback in a way that is arguably closer to human cognition: utilizing precise outcomes rather than just binary rewards. By allowing the

17.01 续段

续段｜目标重标记旨在学习“如何产生某种反馈”；SDPO 用失败反馈判断原目标为何未达成。这是训练用途与语义的不同。

17.02 蒸馏定位

蒸馏定位｜传统蒸馏让学生模仿教师分布；在策略蒸馏让学生在自己的生成前缀上接受监督，以减轻训练与部署分布偏移。

17.03 自蒸馏定位

自蒸馏定位｜先前方法常把额外上下文下生成的新答案做离策略训练；SDPO 训练的是学生自身采样轨迹上的分布差异，目标是纠正自身错误。

17.04 同期工作

同期工作｜也有在网格世界或固定专家数据上用在策略自蒸馏的研究；本文强调在线环境丰富反馈及代码、科学推理实验。

17.05 结论

结论｜提出 RLRF 与 SDPO，通过反馈条件自我教师提供密集信用分配，并可通过修改标准 RLVR 管线实现。

17.06 实验证据

实验证据｜标量环境、丰富反馈编程环境和单题测试时适应均有收益；模型能力越强一般受益越大，仍需结合任务与反馈条件解读。

17.07 意义

意义｜精确的失败证据可转成自监督，而不只依赖“对/错”。关于更接近人类认知的描述属于作者的解释性类比。

model to determine retrospectively how it should have acted, we demonstrate that language models can convert diverse tokenized feedback into effective self-supervision.

18.01 – **Limitations.** Our findings show that SDPO's performance depends on a model's in-context learning ability, suggesting that SDPO is primarily applicable for RL-training stronger base models, while it can underperform GRPO on weaker models. Moreover, performance depends on the quality of the environment feedback. If the environment provides uninformative or misleading feedback, a model may not be able to learn from it through SDPO. Finally, SDPO adds a small computational overhead compared to GRPO for computing the log-probs of the retrospective model. While often negligible, this may be a larger overhead for smaller models with shorter generation lengths, where generation time is comparatively small.

Future Work. Our work highlights several exciting directions for future research:

18.02 – • **Long-horizon and agentic settings.** RLRF is particularly appealing when trajectories are long or expose information about intermediate states. Evaluating SDPO in agentic environments is a natural next step.

18.03 – • **Training dynamics at scale.** Beyond our evaluation on LiveCodeBench, it would be particularly interesting to scale SDPO to large multi-task RL training runs and further study its scaling properties with frontier base models.

18.04 – • **Beyond verifiable rewards.** While we focused on verifiable code generation, many tasks provide textual feedback without a ground-truth verifier. Investigating whether SDPO's retrospection mechanism can improve alignment in open-ended text generation or continuous-reward tasks remains an open empirical question.

18.05 – • **Behavioral differences in reasoning.** We observed that SDPO induces qualitatively different reasoning patterns than GRPO, notably avoiding the latter's tendency toward verbosity and superficial reasoning. Future work should systematically study how individual aspects, such as the reprompt template, influence behavior.

Author Contributions

Jonas Hübotter conceived of the project in summer 2025 and has been working on it full-time since then, leading the team. Jonas proposed the conceptual framework of self-distillation for credit assignment with input from Lejs, implemented the algorithm with help from others, led the quantitative experiments on LCBv6, and led the writing of the paper.

Frederike Lübeck led the design of the code environment, led the design and evaluation of the TTT setting in Section 5 with input from Jonas, contributed to the project direction in discussions, and contributed significantly to the writing of the paper.

Lejs Behric noted the dense credit assignment of knowledge distillation with strong teacher models in discussions with Jonas, inspiring the idea of self-distillation. Further, Lejs led the evaluation of different teacher templates, co-led the development of a tool for qualitative analysis of runs with Marco and Daniel, helped implement parts of the algorithm, and contributed to the project direction in discussions.

Anton Baumann joined in December 2025 and led the evaluation of SDPO without rich feedback in Section 3 with input from Jonas, and contributed to the writing of the paper.

Marco Bagatella and Daniel Marta co-led the development of a tool for qualitative analysis of runs with Lejs, contributed to the training infrastructure, and contributed to the project direction in discussions.

Ido Hakimi significantly contributed to the initial codebase and experimental setup, contributed early algorithmic ideas, and contributed to the project direction in discussions.

Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause supported this project, with Idan and Carlos joining in December 2025. They made significant contributions to the project direction in discussions and gave valuable advice on our presentation. Thomas and Idan, in particular, significantly contributed to the development of core algorithmic ideas and design of experiments. Thomas further evaluated checkpoints

18.01 局限

局限｜弱模型的回顾能力不足时可能逊于 GRPO；无信息或误导反馈会损伤教师质量。额外前向开销在小模型、短生成任务中相对更显著。

18.02 未来工作

未来工作｜在长时程智能体中利用中间状态反馈，检验本文在更复杂交互轨迹上的可扩展性。

18.03 未来工作

未来工作｜扩大到多任务训练和更强基模，研究教师自举的稳定性与规模规律。

18.04 开放问题

开放问题｜无真实验证器的文本反馈可能有噪声与偏差；开放式生成和连续奖励场景的效果尚未被这些实验充分证实。

18.05 行为研究

行为研究｜需系统拆分提示模板、反馈组成与优化目标对简洁推理的影响，验证为何减少冗余而保持效果。

on holdout benchmarks. Carlos suggested the qualitative analysis of reasoning traces in Figure 7 and the presentation of TTT results in Section 5. Andreas pointed out valuable connections to existing work in RL which shaped the direction of the project.

Acknowledgments

We would like to thank Akira Yoshiyama, Yassir Akram, Parnian Kassraie, Jonathan Thomm, Roman Vorushin, Afra Amini, Imanol Schlag, Yu Sun, and Moritz Hardt for helpful discussions. We thank Eduard Durech for helpful conversations regarding the scaling of RL fine-tuning and for his technical guidance on distributed infrastructure and long-context optimization. We are grateful to Ruixu Zhou from Tsinghua University & the Tencent Hunyuan Team for pointing out an error in the initially derived gradient estimator. Furthermore, we would like to thank Leander Diaz-Bone for supporting dataset generation.

This project was supported through the Swiss AI compute grant a156 and, in part, compute grant infra01. JH was supported by the Swiss National Science Foundation under NCCR Automation, grant agreement 51NF40 180545. FL and MB were supported by the ETH-MPI Center for Learning Systems. TKB and IH were supported by an ETH AI Center Postdoctoral Fellowship. DM was supported by the Knut and Alice Wallenberg Foundation.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *ICLR*, 2024.

Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for few-shot learning. In *ICML*, 2025.

Afra Amini, Tim Vieira, and Ryan Cotterell. Better estimation of the kullback–leibler divergence between language models. In *NeurIPS*, 2025.

Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *NeurIPS*, 2017.

Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *NeurIPS*, 2017.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. In *NeurIPS*, 2025.

Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint ArXiv:2005.14165*, 2020.

Bowen Cao, Deng Cai, and Wai Lam. Infiniteicl: Breaking the limit of context window size via long short-term memory transformation. In *ACL*, 2025a.

Meng Cao, Shuyuan Zhang, Xiao-Wen Chang, and Doina Precup. Scar: Shapley credit assignment for more efficient rlhf. *arXiv preprint arXiv:2505.20417*, 2025b.

Alex J Chan, Hao Sun, Samuel Holt, and Mihaela Van Der Schaar. Dense reward for free in reinforcement learning from human feedback. In *ICML*, 2024.

Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025a.

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests. In *ICLR*, 2022.

Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. Retaining by doing: The role of on-policy data in mitigating forgetting. *arXiv preprint arXiv:2510.18874*, 2025b.

Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *NeurIPS*, 2021a.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021b.

Wentse Chen, Jiayu Chen, Fahim Tajwar, Hao Zhu, Xintong Duan, Ruslan Salakhutdinov, and Jeff Schneider. Retrospective in-context learning for temporal credit assignment with large language models. In *NeurIPS*, 2025c.

Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael Jordan, Joseph E Gonzalez, et al. Chatbot arena: An open platform for evaluating llms by human preference. In *ICML*, 2024.

Eunbi Choi, Yongrae Jo, Joel Jang, and Minjoon Seo. Prompt injection: Parameterization of fixed inputs. *arXiv preprint arXiv:2206.11349*, 2022.

Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.

Leander Diaz-Bone, Marco Bagatella, Jonas Hübotter, and Andreas Krause. Discover: Automated curricula for sparse-reward reinforcement learning. In *NeurIPS*, 2025.

Elizabeth D Dolan and Jorge J Moré. Benchmarking optimization software with performance profiles. *Mathematical programming*, 91(2), 2002.

Zi-Yi Dou, Cheng-Fu Yang, Xueqing Wu, Kai-Wei Chang, and Nanyun Peng. Re-rest: Reflection-reinforced self-training for language agents. In *EMNLP*, 2024.

Ahmed El-Kishky, Alexander Wei, Andre Saraiva, Borys Minaiev, Daniel Selsam, David Dohan, Francis Song, Hunter Lightman, Ignasi Clavera, Jakub Pachocki, et al. Competitive programming with large reasoning models. *arXiv preprint arXiv:2502.06807*, 2025.

Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, et al. Cartridges: Lightweight and general-purpose long context representations via self-study. In *ICLR*, 2026.

Kehua Feng, Keyan Ding, Weijie Wang, Xiang Zhuang, Zeyuan Wang, Ming Qin, Yu Zhao, Jianhua Yao, Qiang Zhang, and Huajun Chen. Sciknoweval: Evaluating multi-level scientific knowledge of large language models. *arXiv preprint arXiv:2406.09098*, 2024a.

Xidong Feng, Bo Liu, Yan Song, Haotian Fu, Ziyu Wan, Girish A Koushik, Zhiyuan Hu, Mengyue Yang, Ying Wen, and Jun Wang. Natural language reinforcement learning. *arXiv preprint arXiv:2411.14251*, 2024b.

Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. Rlef: Grounding code llms in execution feedback with reinforcement learning. In *ICML*, 2025.

Prasoon Goyal, Scott Niekum, and Raymond J Mooney. Using natural language for reward shaping in reinforcement learning. In *IJCAI*, 2019.

Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. Bootstrap your own latent-a new approach to self-supervised learning. In *NeurIPS*, 2020.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. 2024.

Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, et al. Openthoughts: Data recipes for reasoning models. In *ICLR*, 2026.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *ICML*, 2018.

Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. In *ICLR*, 2024.

Ali Hatamizadeh, Syeda Nahida Akter, Shrimai Prabhumoye, Jan Kautz, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, and Yejin Choi. Rlp: Reinforcement as a pretraining objective. In *ICLR*, 2026.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-zero: Self-evolving reasoning llm from zero data. In *ICLR*, 2026.

Jonas Hübotter, Patrik Wolf, Alexander Shevchenko, Dennis Jüni, Andreas Krause, and Gil Kur. Specialization after generalization: Towards understanding test-time training in foundation models. In *ICLR*, 2026.

Jonas Hübotter, Sascha Bongni, Ido Hakimi, and Andreas Krause. Efficiently learning at test-time: Active fine-tuning of llms. In *ICLR*, 2025a.

Jonas Hübotter, Leander Diaz-Bone, Ido Hakimi, Andreas Krause, and Moritz Hardt. Learning on the job: Test-time curricula for targeted reinforcement learning. *arXiv preprint arXiv:2510.04786*, 2025b.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *ICLR*, 2025.

Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2), 1998.

Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. Vineppo: Refining credit assignment in rl training of llms. In *ICML*, 2025.

Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for llms. In *ICLR*, 2026.

Yoon Kim and Alexander M Rush. Sequence-level knowledge distillation. In *EMNLP*, 2016.

Kimi, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1.5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.

Kalle Kujanpää, Pekka Marttinen, Harri Valpola, and Alexander Ilin. Efficient knowledge injection in LLMs via self-distillation. *TMLR*, 2025.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *PSIGOPS*, 2023.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. In *COLM*, 2025.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. In *NeurIPS*, 2022.

Kyungjae Lee, Dasol Hwang, Sunghyun Park, Youngsoo Jang, and Moontae Lee. Reinforcement learning from reflective feedback (rlrf): Aligning and improving llms via fine-grained self-reflection. *arXiv preprint arXiv:2403.14238*, 2024.

Yoonho Lee, Joseph Boen, and Chelsea Finn. Feedback descent: Open-ended text optimization via pairwise comparison. *arXiv preprint arXiv:2511.07919*, 2025.

Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.

Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E Gonzalez, and Ion Stoica. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. In *ICML*, 2025a.

Yi-Chen Li, Tian Xu, Yang Yu, Xuqin Zhang, Xiong-Hui Chen, Zhongxiang Ling, Ningjing Chao, Lei Yuan, and Zhi-Hua Zhou. Generalist reward models: Found inside large language models. *arXiv preprint arXiv:2506.23235*, 2025b.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *ICLR*, 2023.

Grace Liu, Michael Tang, and Benjamin Eysenbach. A single goal is all you need: Skills and exploration emerge from contrastive rl without rewards, demonstrations, or subgoals. In *ICLR*, 2025a.

Hao Liu, Carmelo Sferrazza, and Pieter Abbeel. Chain of hindsight aligns language models with feedback. *arXiv preprint arXiv:2302.02676*, 2023.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. In *COLM*, 2025b.

Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. URL https://thinkingmachines.ai/blog/on-policy-distillation.

Renjie Luo, Zichen Liu, Xiangyan Liu, Chao Du, Min Lin, Wenhu Chen, Wei Lu, and Tianyu Pang. Language models can learn from verbal feedback without scalar rewards. *arXiv preprint arXiv:2509.22638*, 2025.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.

Purbesh Mitra and Sennur Ulukus. Semantic soft bootstrapping: Long context reasoning in llms without reinforcement learning. *arXiv preprint arXiv:2512.05105*, 2025.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540), 2015.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori B Hashimoto. s1: Simple test-time scaling. In *EMNLP*, 2025.

Andrew Y Ng, Stuart Russell, et al. Algorithms for inverse reinforcement learning. In *ICML*, 2000.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, et al. Olmo 3. *arXiv preprint arXiv:2512.13961*, 2025.

Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*, 2023.

Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. Fitnets: Hints for thin deep nets. In *ICLR*, 2015.

Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.

Mehrzad Samadi, Aleksander Ficek, Sean Narenthiran, Siddhartha Jain, Wasi Uddin Ahmad, Somshubra Majumdar, Vahid Noroozi, and Boris Ginsburg. Scaling test-time compute to achieve ioi gold medal with open-weight models. *arXiv preprint arXiv:2510.14232*, 2025.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Tom Schaul, Daniel Horgan, Karol Gregor, and David Silver. Universal value function approximators. In *ICML*, 2015.

Jérémy Scheurer, Jon Ander Campos, Tomasz Korbak, Jun Shern Chan, Angelica Chen, Kyunghyun Cho, and Ethan Perez. Training language models with language feedback at scale. *arXiv preprint arXiv:2303.16755*, 2023.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *ICML*, 2015.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *ICLR*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. Rewarding progress: Scaling automated process verifiers for llm reasoning. In *ICLR*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026a.

Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. Rl's razor: Why online reinforcement learning forgets less. In *ICLR*, 2026b.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *EuroSys*, 2025.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529 (7587), 2016.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

Charlie Snell, Dan Klein, and Ruiqi Zhong. Learning by distilling context. *arXiv preprint arXiv:2209.15189*, 2022.

Yuda Song, Lili Chen, Fahim Tajwar, Remi Munos, Deepak Pathak, J Andrew Bagnell, Aarti Singh, and Andrea Zanette. Expanding the capabilities of reinforcement learning via text feedback. *arXiv preprint arXiv:2602.02482*, 2026.

Moritz Stephan, Alexander Khazatsky, Eric Mitchell, Annie S Chen, Sheryl Hsu, Archit Sharma, and Chelsea Finn. Rlvf: Learning from verbal feedback without overgeneralization. In *ICML*, 2024.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *ICML*, 2020.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. Learning to (learn at test time): Rnns with expressive hidden states. In *ICML*, 2025.

Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 1998.

Arnuv Tandon, Karan Dalal, Xinhao Li, Daniel Koceja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, et al. End-to-end test-time training for long context. *arXiv preprint arXiv:2512.23675*, 2025.

Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.

Belen Martin Urcelay, Andreas Krause, and Giorgia Ramponi. From words to rewards: Leveraging natural language for reinforcement learning. In *TMLR*, 2026.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

Hanyang Wang, Lu Wang, Chaoyun Zhang, Tianjun Mao, Si Qin, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. Text2grad: Reinforcement learning from natural language feedback. In *ICLR*, 2026.

Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *ACL*, 2024a.

Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. In *NeurIPS*, 2025.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In *NeurIPS*, 2024b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3), 1992.

Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V Le. Self-training with noisy student improves imagenet classification. In *CVPR*, 2020.

Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2reward: Reward shaping with language models for reinforcement learning. In *ICLR*, 2024.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.

Wenkai Yang, Yankai Lin, Jie Zhou, and Ji-Rong Wen. Distilling rule-based knowledge into large language models. In *COLING*, 2025b.

Zhaorui Yang, Tianyu Pang, Haozhe Feng, Han Wang, Wei Chen, Minfeng Zhu, and Qian Liu. Self-distillation bridges distribution gap in language model fine-tuning. In *ACL*, 2024.

Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your efficient rl framework secretly brings you off-policy rl training, 2025. URL https://fengyao.notion.site/off-policy-rl.

Weiran Yao, Shelby Heinecke, Juan Carlos Niebles, Zhiwei Liu, Yihao Feng, Le Xue, Rithesh Murthy, Zeyuan Chen, Jianguo Zhang, Devansh Arpit, et al. Retroformer: Retrospective large language agents with policy gradient optimization. In *ICLR*, 2024.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. In *NeurIPS*, 2025.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? In *NeurIPS*, 2025.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639:609–616, 2025.

Mert Yuksekgonul, Daniel Koceja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, et al. Learning to discover at test time. *arXiv preprint arXiv:2601.16175*, 2026.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D Goodman. Star: Bootstrapping reasoning with reasoning. In *NeurIPS*, 2022.

Kai Zhang, Xiangchao Chen, Bo Liu, Tianci Xue, Zeyi Liao, Zhihan Liu, Xiyao Wang, Yuting Ning, Zhaorun Chen, Xiaohan Fu, et al. Agent learning via early experience. *arXiv preprint arXiv:2510.08558*, 2025.

Tianjun Zhang, Fangchen Liu, Justin Wong, Pieter Abbeel, and Joseph E Gonzalez. The wisdom of hindsight makes language models better instruction followers. In *ICML*, 2023.

Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. In *NeurIPS*, 2025.

Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025a.

Tianyu Zheng, Tianshun Xing, Qingshui Gu, Taoran Liang, Xingwei Qu, Xin Zhou, Yizhi Li, Zhoufutu Wen, Chenghua Lin, Wenhao Huang, et al. First return, entropy-eliciting explore. *arXiv preprint arXiv:2507.07017*, 2025b.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

Ruiyang Zhou, Shuozhe Li, Amy Zhang, and Liu Leqi. Expo: Unlocking hard reasoning with self-explanation-guided reinforcement learning. In *NeurIPS*, 2025.

Xiangxin Zhou, Zichen Liu, Anya Sims, Haonan Wang, Tianyu Pang, Chongxuan Li, Liang Wang, Min Lin, and Chao Du. Reinforcing general reasoning without verifiers. In *ICLR*, 2026.

Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *AAAI*, 2008.

Yuxin Zuo, Kaiyan Zhang, Shang Qu, Li Sheng, Xuekai Zhu, Biqing Qi, Youbang Sun, Ganqu Cui, Ning Ding, and Bowen Zhou. Ttrl: Test-time reinforcement learning. In *NeurIPS*, 2025.

Contents

1 Introduction **1**

2 SDPO: Self-Distillation Policy Optimization **3**
- 2.1 Comparison to RLVR . . . 5
- 2.2 Compute time & memory . . . 5
- 2.3 Stability improvements . . . 6

3 Learning without Rich Environment Feedback **6**
- 3.1 Experimental setting . . . 6
- 3.2 Results . . . 7
- 3.3 Self-distillation learns to reason concisely . . . 7

4 Learning with Rich Environment Feedback **8**
- 4.1 Self-distillation benefits from stronger models . . . 9
- 4.2 Self-distillation performs dense credit assignment . . . 10
- 4.3 The self-teacher improves during training . . . 10
- 4.4 On-policy self-distillation avoids catastrophic forgetting . . . 11
- 4.5 Can GRPO and SDPO be combined? . . . 12
- 4.6 Which feedback is most informative? . . . 12

5 Solving Hard Questions via Test-Time Self-Distillation **13**
- 5.1 Experimental setting . . . 14
- 5.2 Results . . . 15

6 Related Work **16**
- 6.1 Reinforcement Learning with LLMs . . . 16
- 6.2 Learning from Rich Feedback and through Retrospection . . . 16
- 6.3 Distillation . . . 17
- 6.4 Self-Distillation . . . 17

7 Conclusion, Limitations, and Future Work **17**

A Implementation of SDPO **29**
- A.1 Gradient Estimators . . . 29
- A.2 Regularized teacher . . . 30
- A.3 Approximate Logit Distillation . . . 30
- A.4 Off-Policy Training: Generalization to Logit-Level Losses . . . 31

B Theoretical Analysis **32**

B.1 Proof of Proposition 2.1. 32
B.2 Trust-region Teacher 32

C Additional Related Work **34**
C.1 SDPO as Maximum Entropy RL 34

D Additional Results & Ablations **36**
D.1 Learning without rich environment feedback 36
D.2 Learning with rich environment feedback 37
D.3 Test-time self-distillation 37

E Experiment Details **42**
E.1 Technical setup 42
E.2 Hyperparameters 42
E.3 User Templates 44

F Qualitative Examples **46**
F.1 Visualization of Advantages 46
F.2 Examples 46
F.3 Environment Feedback 49
F.4 Illustrative Example 50

A Implementation of SDPO

29.02 – The following pseudocode in Figure 14 outlines the implementation of SDPO:

29.01 –

```python
def compute_sdpo_loss(batch, teacher_context, loss_mask):
    """
    Computes probabilities of response y under the self-teacher
        and the per-logit SDPO loss.
    """
    # Compute model probabilities for response y
    logprobs_student = compute_log_prob(batch) # (T,V)
    probs_student = logprobs_student.exp() # (T,V)

    # Compute self-teacher probabilities for response y
    teacher_batch = reprompt(batch, teacher_context)
    logprobs_teacher = compute_log_prob(teacher_batch).detach() # (T,V)

    # Compute SDPO loss: per-token divergence
    per_token_loss = divergence(logprobs_student, logprobs_teacher) # (T,)
    return agg_loss(per_token_loss, loss_mask, loss_agg_mode="token-mean")
```

Figure 14: The pseudo-code of SDPO within a standard RL training pipeline. Omitted here is the filtering to top-K logprobs for student and teacher (including a tail term) as described in Appendix A.3. Further, we omit here any importance sampling weights to correct for off-policy data. `reprompt` modifies the batch to incorporate teacher context (i.e., rich feedback). `divergence` implements any per-token divergence such as reverse-KL, forward-KL, or Jensen-Shannon.

In the following, we provide further details on:

- The gradient estimator used in our implementation (Appendix A.1)
- Teacher regularization (Appendix A.2)
- Approximating logit-distillation with the top-K logits for saving GPU memory (Appendix A.3)
- Generalizing PPO-style policy gradient algorithms to logit-level advantages (Appendix A.4)

29.03 – To disambiguate the notation of the self-teacher, we use $q_\theta(\cdot \mid x, f) := \pi_\theta(\cdot \mid \text{reprompt}(x, f))$ in the following. Here, `reprompt` denotes the reprompt template of the self-teacher.

A.1 Gradient Estimators

In this seciton, we discuss two possible gradient estimators for the KL divergence between the current policy $\pi_\theta(y \mid x)$ and the teacher policy $q_\theta(y \mid x, f)$.

29.04 – **Per-token estimator.** Deriving the gradient of the SDPO loss as defined in Equation (1):

$$\mathcal{L}_{\text{token}}(\theta) := \mathbb{E}_{y \sim \text{stopgrad}(\pi_\theta(\cdot|x))}\left[\sum_{t=1}^{T} \text{KL}(\pi_\theta(\cdot \mid x, y_{<t}) \| \text{stopgrad}(\pi_\theta(\cdot \mid x, f, y_{<t})))\right] \tag{5}$$

leads to the following estimator (see a detailed proof in Appendix B.1), which corresponds to the sum of gradients of the KL divergence at each token:

$$\nabla \mathcal{L}_{\text{token}}(\theta) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)}\left[\sum_{t=1}^{T} \mathbb{E}_{\hat{y}_t \sim \pi_\theta(\cdot|x, y_{<t})}\left[\nabla_\theta \log \pi_\theta(\hat{y}_t \mid x, y_{<t}) \cdot \log \frac{\pi_\theta(\hat{y}_t \mid x, y_{<t})}{\pi_\theta(\hat{y}_t \mid x, f, y_{<t})}\right]\right]. \tag{6}$$

This corresponds to the estimator presented in Proposition 2.1. This gradient estimator effectively assumes that the sampling distribution generating y is fixed.

29.01 实现

实现 | 先算学生 log-prob，再对反馈增强提示算教师 log-prob 并 detach，逐位置求散度、按有效响应词元掩码聚合。代码块是论文伪代码，不是对本任务的操作指令。

29.02 补充

补充 | 伪代码省略 top-K 尾项与离策略校正；divergence 可选 reverse-KL、forward-KL 或 JS。复现时必须结合后续附录和各实验超参数。

29.03 记号

记号 | qθ 表示反馈重提示后的教师分布；πθ 是仅以问题和响应前缀为条件的学生。共享参数并不使两个条件分布相同。

29.04 公式 (5)—(6)

公式 (5)—(6) | 采样得到的前缀分布停止梯度，只对该前缀下的 token KL 求导。这是命题 2.1 所用估计器，不是对完整轨迹分布所有依赖求导。

30.01 – **Sequence-level estimator.** An alternative self-distillation objective minimizes the sequence-level KL divergence between student and self-teacher, i.e.,

$$\begin{aligned}\mathcal{L}_{\text{seq}}(\theta) := \mathrm{KL}(\pi_\theta \| q_\theta) &= \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\log \frac{\pi_\theta(y \mid x)}{q_\theta(y \mid x, f)}\right] \\ &= \sum_{t=1}^{T} \mathbb{E}_{s_t\sim\Pi_\theta}\left[\mathrm{KL}(\pi_\theta(\cdot \mid s_t)\|q_\theta(\cdot \mid s_t, f))\right],\end{aligned} \tag{7}$$

where $s_t = (x, y_{<t})$ is the prefix ("state") at step t and Π_θ denotes the prefix distribution under policy π_θ. Estimating the gradient of this objective additionally takes into account how the choice of y_t influences future states $y_{>t}$ (due to the additional dependence on Π_θ).

Amini et al. (2025) show that the corresponding gradient estimator is given by

$$\nabla\mathcal{L}_{\text{seq}}(\theta) = \nabla\mathcal{L}_{\text{token}}(\theta) + \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\sum_{t=1}^{T} \mathrm{KL}(\pi_\theta(\cdot \mid s_t)\|q_\theta(\cdot \mid s_t, f))\nabla_\theta \log \Pi_\theta(s_t)\right]. \tag{8}$$

30.02 – The additional term of the sequence-level gradient captures how prefixes influence the self-distillation divergence of future tokens. We also experimented with this sequence-level gradient estimator but did not find measurable gains relative to its additional complexity.

A.2 Regularized teacher

30.03 – In contrast to standard distillation, the teacher in SDPO changes throughout training. This bootstrapping enables the teacher to improve, but it may also lead to training instability. To stabilize training, we seek to prevent the teacher q from quickly diverging from the initial
30.04 – teacher $q_{\theta_{\text{ref}}}$. We can achieve this by placing an explicit trust-region constraint on q (Schulman et al., 2015; Peng et al., 2019), that is:

$$\sum_t \mathrm{KL}\left(q(y_t \mid x, f, y_{<t})\|q_{\theta_{\text{ref}}}(y_t \mid x, f, y_{<t})\right) \le \epsilon, \quad \epsilon > 0. \tag{9}$$

This trust-region can be implemented in two ways:

1. **Explicit trust-region:** We can define the teacher as the policy closest to q_θ while satisfying the trust-region constraint. This teacher can be expressed as

$$q(y_t \mid x, f, y_{<t}) \propto \exp\left((1-\alpha)\log q_{\theta_{\text{ref}}}(y_t \mid x, f, y_{<t}) + \alpha \log q_\theta(y_t \mid x, f, y_{<t})\right), \tag{10}$$

   with $\alpha \in (0, 1)$ the inverse Lagrange multiplier for the trust-region constraint. We include a full derivation in Appendix B.2. We can plug this explicitly constrained teacher directly into the SDPO objective.

30.05 – 2. **Exponential moving average (EMA):** Alternatively, we can stabilize the teacher's parameters directly; parameterizing $q_{\theta'}$ by θ' and updating as $\theta' \leftarrow (1-\alpha)\theta' + \alpha\theta$ with $\alpha \in (0, 1)$.

30.06 – Note that each implementation has a different practical advantage: The EMA teacher requires additional GPU memory for θ' yet does not introduce any runtime overhead. In contrast, the trust-region teacher requires an additional log-prob computation with $q_{\theta_{\text{ref}}}$ yet does not require additional GPU memory if θ_{ref} is used for explicit KL regularization.

A.3 Approximate Logit Distillation

30.07 – To save GPU memory, we perform distillation only on the top-K tokens predicted by the student:

30.01 公式 (7)—(8)

公式 (7)—(8) | 完整序列 KL 的梯度还需考虑当前 token 如何影响后续前缀分布 Πθ，多出 KL×□logΠθ 项。它不同于第 4.2 节"把优势平均成单一序列数值"的消融。

30.02 发现

发现 | 额外项处理早期动作对未来蒸馏差异的影响；作者尝试后未发现足以抵消复杂度的增益，因此主方法使用固定采样前缀的逐 token 估计器。

30.03 教师稳定性

教师稳定性 | 教师同步变强有利于自举，也可能漂移过快；对它相对初始反馈教师的 KL 加约束以控制这一风险。

30.04 显式教师

显式教师 | 在 log 概率空间混合参考教师与当前教师，再归一化；α 决定当前教师的影响，不是直接混合模型权重。

30.05 EMA 教师

EMA 教师 | 对模型参数做指数平滑，属于参数空间更新。它与上面的分布插值在数学上并不等价，只是两个稳定训练的方案。

30.06 资源取舍

资源取舍 | EMA 需存额外教师参数；显式信赖域需额外参考教师重算。若已有 KL 参考模型，后者可复用，显存成本取决于实现。

30.07 近似蒸馏

近似蒸馏 | 学生 top-K 决定候选集合；尾概率必须保留，不能只把头部概率直接当完整归一化词表。

$$\begin{aligned}\mathcal{L}_{\text{SDPO}}(\theta) &= \sum_{t=1}^{T} \mathrm{KL}(\pi_\theta(\cdot \mid x, y_{<t}) \| \mathrm{stopgrad}(q_\theta(\cdot \mid x, f, y_{<t}))) \\ &\approx \sum_{t=1}^{T} \sum_{\hat{y}_t \in \mathrm{top}_K(\pi_\theta)} \pi_\theta(\hat{y}_t \mid x, y_{<t}) \cdot \log \frac{\pi_\theta(\hat{y}_t \mid x, y_{<t})}{\mathrm{stopgrad}(q_\theta(\hat{y}_t \mid x, f, y_{<t}))} \\ &\quad + \underbrace{\left(1 - \textstyle\sum_{\hat{y}_t \in \mathrm{top}_K(\pi_\theta)} \pi_\theta(\hat{y}_t \mid x, y_{<t})\right) \cdot \log \frac{1 - \sum_{\hat{y}_t \in \mathrm{top}_K(\pi_\theta)} \pi_\theta(\hat{y}_t \mid x, y_{<t})}{\mathrm{stopgrad}\left(1 - \sum_{\hat{y}_t \in \mathrm{top}_K(\pi_\theta)} q_\theta(\hat{y}_t \mid x, f, y_{<t})\right)}}_{\text{tail}}\end{aligned} \tag{11}$$

31.01 – Here, the top-K is with respect to student. Without top-K distillation, we would have to keep two copies of logits in memory: one for teacher and student each. Top-K distillation avoids virtually any memory overhead without impacting performance significantly, since most tokens of the vocabulary are not informative at a given time.

A.4 Off-Policy Training: Generalization to Logit-Level Losses

31.02 – PPO-style clipping (Schulman et al., 2017) with truncated importance sampling (Yao et al., 2025), clip-higher (Yu et al., 2025), fixed length normalization (Liu et al., 2025b):

$$\mathcal{L}_{\text{token}}(\theta) := -\frac{1}{\sum_{i=1}^{G} |y_i|} \sum_{i=1}^{G} \sum_{t=1}^{|y_i|} \min\left(w_{i,t}^{\text{TIS}}, \rho\right) \min\left(w_{i,t} A_{i,t}, \mathrm{clip}(w_{i,t}, 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}) A_{i,t}\right), \tag{12}$$

with $w_{i,t} := \frac{\pi_\theta(y_{i,t}|x,y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x,y_{i,<t})}$, $w_{i,t}^{\text{TIS}} := \frac{\pi_{\theta_{\text{old}}}(y_{i,t}|x,y_{i,<t})}{\pi_{\theta_{\text{old}}}^{\text{rollout}}(y_{i,t}|x,y_{i,<t})}$, and $A_{i,t}$ denotes the per-token advantage.

31.03 – We extend this to a logit-level loss:

$$\begin{aligned}\mathcal{L}_{\text{logit}}(\theta) := -\frac{1}{\sum_{i=1}^{G} |y_i|} \sum_{i=1}^{G} \sum_{t=1}^{|y_i|} \sum_{\hat{y}_{i,t}} &\min\left(\pi_{\theta_{\text{old}}}(\hat{y}_{i,t} \mid x, y_{i,<t}), \rho \pi_{\theta_{\text{old}}}^{\text{rollout}}(\hat{y}_{i,t} \mid x, y_{i,<t})\right) \\ &\min\left(w_{i,t}(\hat{y}_{i,t}) A_{i,t}(\hat{y}_{i,t}), \mathrm{clip}(w_{i,t}(\hat{y}_{i,t}), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}) A_{i,t}(\hat{y}_{i,t})\right),\end{aligned} \tag{13}$$

where $\hat{y}_{i,t}$ sums over all possible tokens at position t for rollout i (or the K most likely under $\pi_{\theta_{\text{old}}}$, cf. Appendix A.3). The TIS changes since we explicitly weight each logit by its probability under $\pi_{\theta_{\text{old}}}$ rather than relying on a Monte Carlo estimate of the expectation over next-token predictions. Here, $A_{i,t}(\hat{y}_{i,t})$ is a per-logit advantage.

31.04 – In our experiments for SDPO, we apply the TIS term on a token-level rather than logit-level.

31.01 公式 (11)

公式 (11) | 学生与教师在相同 top-K 集合上比较，集合外概率合成一个 tail 类别。这是粗粒化分布 KL；丢弃尾部内部差异，因而属于近似而非完整 KL。

31.02 公式 (12)

公式 (12) | 基线包括 PPO 裁剪、截断重要性采样、较大上界裁剪和固定长度归一化。分清旧训练策略、实际 rollout 策略与当前更新策略的概率比。

31.03 公式 (13)

公式 (13) | 把采样 token 的代理目标推广到候选词表求和，用旧策略概率加权；优势和裁剪比率都依赖具体候选词元。

31.04 实现边界

实现边界 | 论文 SDPO 实验把 TIS 校正在 token 层面应用，未完全按分布级逐候选实现；区分推导的通式与实际配置。

B Theoretical Analysis

This section is organized as follows:

- Appendix B.1 derives the SDPO gradient from Proposition 2.1.
- Appendix B.2 derives the trust-region regularized teacher discussed in Appendix A.2.

To disambiguate the notation of the self-teacher, we use $q_\theta(\cdot \mid x, f) := \pi_\theta(\cdot \mid \text{reprompt}(x, f))$ in the following. Here, `reprompt` denotes the reprompt template of the self-teacher.

B.1 Proof of Proposition 2.1.

Proof. In the following, we derive the gradient of $\mathcal{L}_{\text{SDPO}}$.

$$\begin{aligned}\nabla_\theta \mathcal{L}_{\text{SDPO}}(\theta) &= \nabla_\theta \sum_{t=1}^{T} \mathrm{KL}(\pi_\theta(\cdot \mid x, y_{<t}) \| \text{stopgrad}(q_\theta(\cdot \mid x, f, y_{<t}))) \\ &= \nabla_\theta \sum_{t=1}^{T} \sum_{\hat{y}_t} \pi_\theta(\hat{y}_t \mid x, y_{<t}) \log\left(\frac{\pi_\theta(\hat{y}_t \mid x, y_{<t})}{\text{stopgrad}(q_\theta(\hat{y}_t \mid x, f, y_{<t}))}\right)\end{aligned}$$

Let $A_{t,k} := \log\left(\frac{\text{stopgrad}(q_\theta(\hat{y}_t|x,f,y_{<t}))}{\pi_\theta(\hat{y}_t|x,y_{<t})}\right)$. Then,

$$\begin{aligned}&= -\nabla_\theta \sum_{t=1}^{T} \sum_{\hat{y}_t} \pi_\theta(\hat{y}_t \mid x, y_{<t}) A_{t,k} \\ &= -\sum_{t=1}^{T} \sum_{\hat{y}_t} \pi_\theta(\hat{y}_t \mid x, y_{<t}) \nabla_\theta A_{t,k} + A_{t,k} \nabla_\theta \pi_\theta(\hat{y}_t \mid x, y_{<t}).\end{aligned}$$

We have that $\nabla_\theta A_{t,k} = -\nabla_\theta \log \pi_\theta(\hat{y}_t \mid x, y_{<t})$ is the negative score function. Using the score trick, $\pi_\theta(\hat{y}_t \mid x, y_{<t}) \nabla_\theta \log \pi_\theta(\hat{y}_t \mid x, y_{<t}) = \nabla_\theta \pi_\theta(\hat{y}_t \mid x, y_{<t})$. Hence, the first term simplifies to

$$-\sum_{t=1}^{T} \sum_{\hat{y}_t} \pi_\theta(\hat{y}_t \mid x, y_{<t}) \nabla_\theta A_{t,k} = \sum_{t=1}^{T} \sum_{\hat{y}_t} \nabla_\theta \pi_\theta(\hat{y}_t \mid x, y_{<t}) = \sum_{t=1}^{T} \nabla_\theta \underbrace{\sum_{\hat{y}_t} \pi_\theta(\hat{y}_t \mid x, y_{<t})}_{=1} = 0.$$

Thus, the gradient of $\mathcal{L}_{\text{SDPO}}$ is

$$\begin{aligned}\nabla_\theta \mathcal{L}_{\text{SDPO}} &= -\sum_{t=1}^{T} \sum_{\hat{y}_t} A_{t,k} \nabla_\theta \pi_\theta(\hat{y}_t \mid x, y_{<t}) \\ &= -\sum_{t=1}^{T} \sum_{\hat{y}_t} \pi_\theta(\hat{y}_t \mid x, y_{<t}) \left(A_{t,k} \nabla_\theta \log \pi_\theta(\hat{y}_t \mid x, y_{<t})\right) \\ &= -\sum_{t=1}^{T} \mathbb{E}_{\hat{y}_t \sim \pi_\theta(\cdot|x,y_{<t})} \left[A_{t,k} \nabla_\theta \log \pi_\theta(\hat{y}_t \mid x, y_{<t})\right].\end{aligned}$$

□

Notably, the above implies that the gradient of $\mathcal{L}_{\text{SDPO}}$ is equivalent to the gradient of the loss if $A_{t,k} = \text{stopgrad}\left(\log \frac{q_\theta(y_t|x,f,y_{<t})}{\pi_\theta(y_t|x,y_{<t})}\right)$.

B.2 Trust-region Teacher

To stabilize training, we seek to prevent the teacher q from diverging from the initial teacher $q_{\theta_{\text{ref}}}$. We can achieve this by placing an explicit trust-region constraint on the teacher q (Schulman et al., 2015; Peng et al., 2019), that is:

$$\sum_t \mathrm{KL}(q(y_t \mid x, f, y_{<t}) \| q_{\theta_{\text{ref}}}(y_t \mid x, f, y_{<t})) \le \epsilon, \quad \epsilon > 0. \tag{14}$$

32.01 推导路线

推导路线 | 在固定前缀对 KL(p||stopgrad(q)) 求导；令 A=log(q/p)，先展开乘积，再用概率归一化消去额外项。

32.02 关键等式

关键等式 | ∇A=−∇log p；Σa p(a)∇log p(a)=∇Σa p(a)=0。因此求导时出现的额外项消失，而非任意忽略。

32.03 结论

结论 | ∇L=−Σt E[A∇log p]。若用策略梯度代理损失，A 也应停止梯度，避免重复引入 A 对学生参数的导数。

32.04 教师约束

教师约束 | 式 (14) 限制反馈教师 q 与初始反馈教师 q_ref 的逐位置 KL 和；参考对象仍是带反馈的教师，而非不看反馈的学生。

33.01 – In the following, we derive a teacher q which satisfies the trust-region constraint while staying close to the target q_θ. The following optimization problem characterizes such a q (Peng et al., 2019):

$$\begin{aligned} \arg\max_{q \in \Delta} & \sum_t \sum_{y_t} q(y_t \mid x, f, y_{<t}) \log \frac{q_\theta(y_t \mid x, f, y_{<t})}{q_{\theta_{\text{ref}}}(y_t \mid x, f, y_{<t})} \\ \text{s.t.} & \sum_t \mathrm{KL}\big(q(y_t \mid x, f, y_{<t}) \| q_{\theta_{\text{ref}}}(y_t \mid x, f, y_{<t})\big) \le \epsilon, \end{aligned} \tag{15}$$

where Δ denotes the probability simplex. Intuitively, the solution is the q satisfying the trust-region constraint, which is closest to q_θ (i.e., has minimal cross-entropy to q_θ) while being farthest from $q_{\theta_{\text{ref}}}$ (i.e., has maximal cross-entropy to $q_{\theta_{\text{ref}}}$).

33.02 – **Proposition B.1.** *The solution to Equation* (15) *can be expressed in closed form as*

$$q^*(y_t \mid x, f, y_{<t}) \propto \exp\big((1-\alpha) \log q_{\theta_{\text{ref}}}(y_t \mid x, f, y_{<t}) + \alpha \log q_\theta(y_t \mid x, f, y_{<t})\big). \tag{16}$$

Proof. To simplify notation, we omit the conditioning in the following. The Lagrangian (with $\lambda \ge 0$ for the KL constraint and ν for normalization) is

$$\mathcal{L}(q, \lambda, \nu) = \sum_t \sum_{y_t} q(y_t) \log \frac{q_\theta(y_t)}{q_{\theta_{\text{ref}}}(y_t)} - \lambda\Big(\sum_{y_t} q(y_t) \log \frac{q(y_t)}{q_{\theta_{\text{ref}}}(y_t)} - \epsilon\Big) + \nu\Big(\sum_{y_t} q(y_t) - 1\Big).$$

33.03 – Stationarity gives, for all y_t,

$$0 = \frac{\partial \mathcal{L}}{\partial q(y_t)} = \log \frac{q_\theta(y_t)}{q_{\theta_{\text{ref}}}(y_t)} - \lambda\Big(\log \frac{q(y_t)}{q_{\theta_{\text{ref}}}(y_t)} + 1\Big) + \nu.$$

Let $\alpha := 1/\lambda$. Then, the solution to Equation (15) can be characterized in closed form as

$$\begin{aligned} q^*(y_t) &\propto q_{\theta_{\text{ref}}}(y_t) \exp\Big(\alpha \log \tfrac{q_\theta(y_t)}{q_{\theta_{\text{ref}}}(y_t)}\Big) \\ &\propto \exp\big((1-\alpha) \log q_{\theta_{\text{ref}}}(y_t) + \alpha \log q_\theta(y_t)\big). \end{aligned}$$

□

33.04 – Chen et al. (2025c) perform a similar derivation, but use reference $\pi_{\theta_{\text{ref}}}$, which we observe to underperform compared to the reference $q_{\theta_{\text{ref}}}$.

33.01 约束优化

约束优化 | 在不偏离参考教师太远的条件下，提高当前教师相对参考教师更偏好词元的权重。目标是期望 log(qθ/qref)，不应简单说成直接最小化到 qθ 的 KL。

33.02 闭式解

闭式解 | q*∝qref^(1−α)qθ^α。先混合对数概率再归一化，可理解为几何插值；α=1/λ，约束越紧通常越靠近参考教师。

33.03 证明

证明 | 给 KL 约束和概率和为 1 分别加拉格朗日乘子，对 q 求驻点，可将 log q 写成两个教师 log 概率的线性组合。

33.04 比较

比较 | 作者报告参考反馈教师 q_ref 优于参考学生 π_ref。这里应关注参考分布是否同样条件化了反馈，而不只是参数是否冻结。

C Additional Related Work

34.01 – **Value networks and Monte Carlo advantage estimation.** Several prior approaches aim to improve credit assignment but face the same information bottleneck as GRPO. Classical RL frequently trains value networks which provide token-level advantages, but themselves are learned from scalar rewards (Schulman et al., 2016; 2017). Furthermore, value networks incur significant computational and memory overhead and are therefore typically not used to train LLMs. Other recent work estimates token-level advantages by performing additional generations starting from various positions in the original attempt (Kazemnejad et al., 2025; Zheng et al., 2025b). While this can learn with fewer gradient steps than GRPO it still uses only scalar rewards as signal and requires costly additional generations.

34.02 – **Dense credit assignment with a reward model.** Several recent works study dense (per-token) reward assignment given access to an external reward model, typically by exploiting the reward model's internal structure (Chan et al., 2024; Cao et al., 2025b). Relatedly, Li et al. (2025b) argue that a token-level reward signal is implicit in an LLM's logits by linking next-token prediction to offline inverse reinforcement learning, effectively yielding a training-free reward model for RL fine-tuning.

34.03 – **Partial observability.** From the perspective of classical RL, many verifiable domains for LLMs are naturally *partially observable*: executing a proposed solution induces a latent environment state (e.g., failing tests or states of an agentic system) that is revealed only through rich feedback. This aligns with the formalism of partially observable Markov decision processes (POMDPs), where agents must act under incomplete observations of state (Kaelbling et al., 1998; Sutton & Barto, 1998). By contrast, RLVR and RLHF pipelines typically discard this observation channel and learn only from terminal scalar rewards or pairwise preferences.

34.04 – **Relation to test-time training.** Our setting from Section 5 can be seen as a special case of test-time training where the model itself is updated at test-time using self-distillation. Updating the model at test-time is known as test-time training (Sun et al., 2020; 2025; Hardt & Sun, 2024; Hübotter et al., 2025a;b; Akyürek et al., 2025; Behrouz et al., 2025; Tandon et al., 2025; Hübotter et al., 2026). Unlike prior work, self-distillation uses the in-context learning ability of the current model to attribute credit after receiving feedback. This can be seen as simulating long-context reasoning with periodic compression of context into the model weights.

C.1 SDPO as Maximum Entropy RL

34.05 – The SDPO objective resembles the objective in maximum entropy RL (e.g., Levine, 2018; Haarnoja et al., 2018) with a particular choice of reward function.

Maximum Entropy RL Consider optimizing

$$\arg\max_{\theta} \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\sum_t r(y_t \mid x, y_{<t})\right] + \lambda \mathrm{H}[\pi_\theta(\cdot \mid x)], \quad \lambda > 0 \tag{17}$$

where $\pi_\theta(y \mid x) = \prod_{t=1}^{T} \pi_\theta(y_t \mid x, y_{<t})$ and $\mathrm{H}[\pi_\theta(\cdot \mid x)] = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[-\log \pi_\theta(y \mid x)]$ is the entropy of the policy. Here, $r(y_t \mid x, y_{<t})$ is an arbitrary reward function, possibly "dense" (i.e., per-token). Equation (17) is known as maximum entropy RL. It is known that this objective is equivalent to solving a variational inference problem which discuss next.

34.06 – To this end, we define a Bernoulli random variable $\mathcal{C}$ which is 1 if the attempt y is correct and 0 otherwise. We then define its distribution as $p(\mathcal{C} = 1 \mid x, y) \propto \exp(\frac{1}{\lambda}\sum_t r(y_t \mid x, y_{<t}))$. Further assuming w.l.o.g. that the "prior" over responses is uniform, we can express the posterior conditioned on the event of correctness as

$$\pi^\star(y \mid x) := p(y \mid x, \mathcal{C} = 1) \propto p(\mathcal{C} = 1 \mid x, y) \propto \exp\left(\frac{1}{\lambda}\sum_t r(y_t \mid x, y_{<t})\right). \tag{18}$$

34.01 相关方法

相关方法 | 价值网络或额外前缀 rollout 可改善信用分配，但仍主要从标量回报估计，并增加模型或采样成本。

34.02 奖励模型路线

奖励模型路线 | 外部奖励模型可产生逐 token 信号，常需利用其内部结构；SDPO 则通过反馈条件模型直接比较分布。

34.03 状态解释

状态解释 | 运行结果反映原本不可见的环境状态；从 POMDP 视角看，文本反馈是新的观测通道，单一结果奖励会丢掉它。

34.04 测试时训练

测试时训练 | 第 5 节直接修改权重；可看成定期把长上下文经验压入参数，区别于固定模型只延长提示。

34.05 最大熵联系

最大熵联系 | 式 (17) 同时鼓励奖励与策略熵。作者用正确性事件构造指数形式的后验分布，连接 RL 目标与 KL 最小化。

34.06 公式解读

公式解读 | 假设正确性似然与 exp(总奖励/λ) 成比例且先验均匀，则正确性条件下的理想回答分布呈 Gibbs 形式；这些是假设下的理论对应。

35.01 – Then, Equation (17) is equivalent to minimizing the KL divergence with respect to $\pi^\star$:

$$\arg\min_\theta \sum_t \mathrm{KL}(\pi_\theta(y_t \mid x, y_{<t}) \| \pi^\star(y_t \mid x, y_{<t})). \tag{19}$$

35.02 – **SDPO optimizes an implicit reward defined by the teacher** Note that Equation (19) is equivalent to the SDPO objective (Equation (1)) with implicit reward $r(y_t \mid x, y_{<t}) = \log q(y_t \mid x, f, y_{<t})$ and $\lambda = 1$. In this sense, SDPO can be seen as a maximum entropy RL algorithm with dense rewards constructed implicitly through the retrospective model.

35.03 – This also points to a connection of SDPO to inverse RL (Ng et al., 2000; Ziebart et al., 2008; Rafailov et al., 2023), where the goal is to recover an unknown reward function. In SDPO, the student learns an implicit reward function defined by the retrospective model.

35.01 等价解释

等价解释 | 在合适前缀期望与条件分解下，最大熵策略目标对应向理想后验分布逼近。不要把固定前缀训练误认为完整序列目标的全部梯度。

35.02 隐式奖励

隐式奖励 | 作者以 log q 作为逐 token 奖励解释 SDPO，λ=1。教师给出的是反馈导出的代理学习信号，不等于恢复了真实环境奖励。

35.03 逆 RL 联系

逆 RL 联系 | 可视为从事后模型中得到隐式奖励，再优化学生。此处是目标结构的联系，不是另行训练一个显式逆强化学习模型。

D Additional Results & Ablations

This section is organized as follows:

- Appendix D.1 contains results and ablations for Section 3.
- Appendix D.2 contains results and ablations for Section 4.
- Appendix D.3 contains results and ablations for Section 5.

D.1 Learning without rich environment feedback

36.01 – • Table 7 reports results when optimal hyperparameters are selected for each model/task combination.

36.02 – • Table 8 compares average response lengths of SDPO and GRPO.

	Chemistry		Physics		Biology		Materials		Tool use	
	1h	5h	1h	5h	1h	5h	1h	5h	1h	5h
Qwen3-8B	41.2		59.2		30.8		58.9		57.5	
+ GRPO	65.9	74.5	62.9	74.5	35.1	**59.9**	**74.3**	77.1	**61.7**	68.1
+ GRPO (on-policy)	52.2	71.6	62.9	74.8	49.8	49.8	73.3	75.8	**61.7**	68.1
+ **SDPO** (on-policy)	**73.2**	**80.9**	**70.6**	**80.6**	**50.6**	56.8	72.1	**78.3**	56.4	**68.5**
Olmo3-7B-Instruct	22.8		37.7		16.2		36.7		39.3	
+ GRPO	53.1	67.7	55.3	63.3	35.6	**55.8**	73.8	78.1	56.4	**65.0**
+ GRPO (on-policy)	47.1	65.4	**62.7**	62.7	**49.8**	49.8	67.9	74.4	56.0	61.3
+ **SDPO** (on-policy)	**68.0**	**80.0**	60.3	**71.4**	48.0	52.8	**75.3**	**79.2**	**57.3**	62.5

Table 7: **Comparison of SDPO and GRPO on reasoning-related benchmarks.** We report the highest achieved avg@16 within 1 hour and 5 hours of wall-clock training time, respectively. Both SDPO and on-policy GRPO perform one gradient step per generation batch, while GRPO performs 4 off-policy mini batch steps. We select optimal hyperparameters for SDPO and baselines based on 5h accuracy. We perform this selection independently for each model and dataset. Each run is performed on a node with 4 NVIDIA GH200 GPUs. Together with initialization and validation, each run takes approximately 6 hours. *As opposed to Table 3 which selects globally optimal hyperparameters per method, this table selects optimal hyperparameters individually for each model/task combination based on 5h accuracy.* The hyperparameter grid is described in Section E.2.1.

Model	GRPO	SDPO	Reduction of SDPO
Qwen3-8B	820.8	255.8	3.2×
Olmo3-7B-Instruct	1095.4	343.9	3.2×

Table 8: Average response lengths of SDPO and GRPO (averaged across tasks from Section 3). Both algorithms are evaluated in the on-policy setting.

中文精读 · Wang | 原文第 36 页

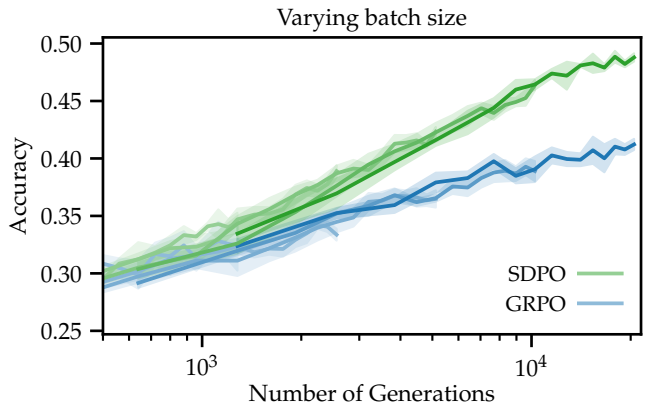
36.01 表 7

表 7 | 每个模型/任务组合单独选择最优超参数；主表 3 是跨模型任务统一选择。对照时注意调参粒度差异，不把结果差异直接归因于算法。

36.02 表 8

表 8 | 在相同 on-policy 条件下，两种模型跨任务平均回答长度均约缩短 3.2 倍；比单个化学任务的 11 倍更能反映总体趋势。

蓝色数字对应段落 / 图表；术语在首次出现处批注



37.01 – Figure 16: Accuracy (pass@1) for varying train batch sizes (4, 8, 16, 32) and number of rollouts (4, 8) for training SDPO and GRPO with Qwen3-8B (Yang et al., 2025a) on LCBv6, ± stderr across 3 seeds. Different shades of the same color correspond to different runs.

D.2 Learning with rich environment feedback

D.2.1 Additional Results

37.02 – Figure 15 shows the average accuracy of SDPO and GRPO stratified by question difficulty. LCB differentiates between easy, medium, and hard questions. As displayed, SDPO significantly improves over GRPO in solving medium and hard questions, highlighting the importance of rich feedback for challenging tasks. Note that this categorization of questions is different from the one in Section 5.

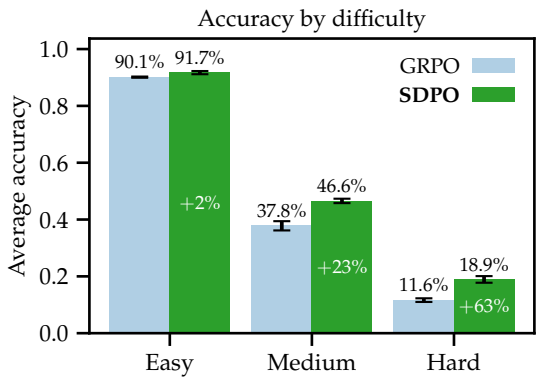


Figure 15: Average accuracy during training until step 80, stratified by difficulty. Error bars show standard deviation across 3 seeds.

In Figure 16, we compare different train batch sizes and number of rollouts for training GRPO and SDPO on LCBv6.

37.05 – Complementing the results shown in Figure 8, we show additional results using Qwen2.5-Instruct (Qwen et al., 2024) in Figure 17.

D.2.2 Training Stability

37.03 – Figure 18 shows diverse metrics logged during training, including the loss, entropy, average gradient norm, and average response length.

D.2.3 Baselines

37.04 – Table 9 compares the performance on LCBv6 of various baselines, including two variants of GRPO, GSPO, and CISPO to SDPO.

D.3 Test-time self-distillation

Complementing the results shown in Section 5, we show the discovery@k curves for all hard question in Figure 20, and report the mean number of generations until the first discovery in Table 10. Further, Table 11 shows the per-question accuracy of the self-teacher at the initial training step of SDPO. In Figure 19, we ablate the choice of batch size for SDPO and the in-context reprompting strategy for multi-turn sampling.

37.01 图 16

图 16 | 改变问题批量和每题 rollout 数，检验样本效率趋势是否依赖单一批量配置。要按生成总数比较，不能只比较更新步数。

37.02 难度分组

难度分组 | SDPO 对中等/困难代码题收益较大，简单题两者接近。这里的 easy/medium/hard 是 LCB 分类，与第 5 节按 pass@64 选题不同。

37.03 诊断入口

诊断入口 | 图 18 给出损失、熵、梯度范数和回答长度；这些用于观察优化过程，不单独构成准确率改善的证据。

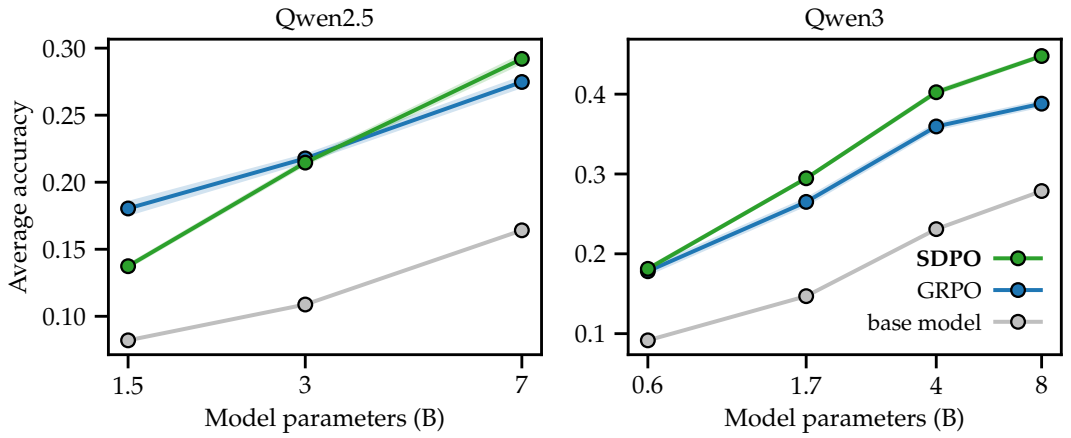
37.04 基线入口

基线入口 | 图表将 SDPO 与 GSPO、CISPO 和 GRPO 变体比较，后页报告最终/训练期间平均代码准确率。

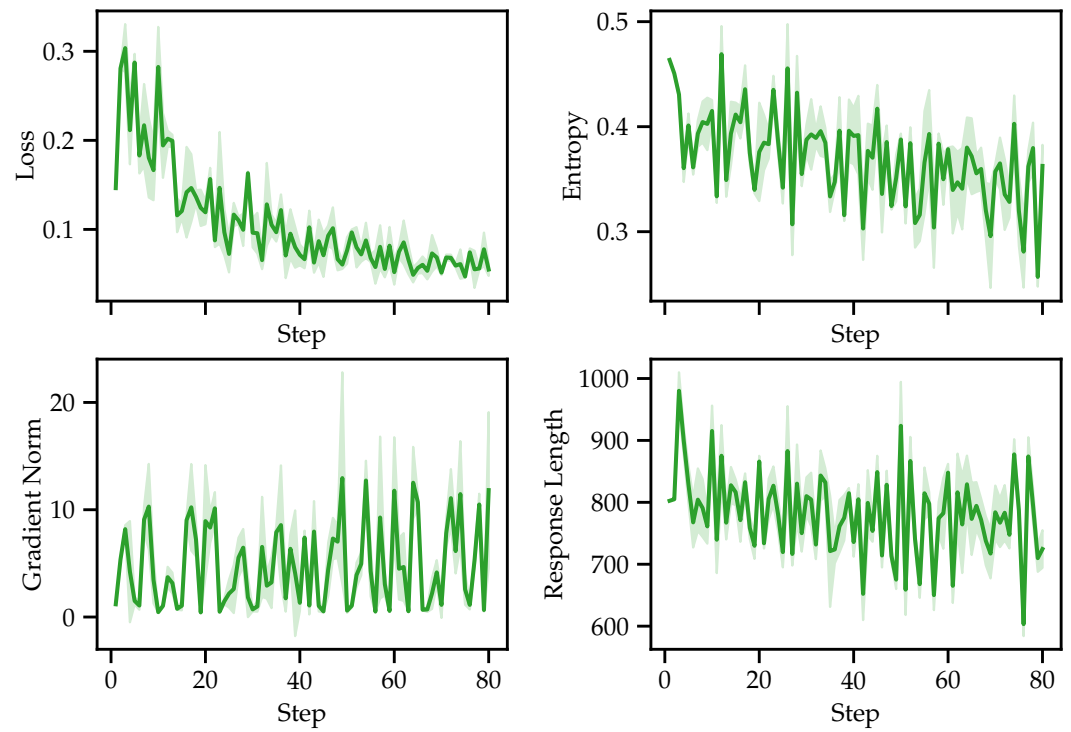
37.05 测试时附录

测试时附录 | 逐题发现曲线、截断平均首次成功次数、初始教师准确率，以及批量和多轮模板消融，用于核对主文总体曲线。

蓝色数字对应段落 / 图表；术语在首次出现处批注



38.01 – Figure 17: Average validation accuracy by model size, ± std across 3 seeds. With Qwen2.5-Instruct (Qwen et al., 2024) and Qwen3 (Yang et al., 2025a) on LCBv6. Until step 65 for Qwen2.5 and until step 80 for Qwen3.



38.02 – Figure 18: Loss, entropy, avg. gradient norm and avg. response length during training of SDPO on LCBv6 (Section 4

.

38.03 – In the selection of hard questions, we have discarded one malformed question (Q9) where the coding environment did not correctly validate the solution due to rounding inaccuracies, which led to failures even with correct logic.

38.01 图 17

图 17 | Qwen2.5 与 Qwen3 采用不同截止训练步数，图中是训练期间平均验证准确率。它支持规模相关趋势，但不能把所有跨系列差异仅归因于参数量。

38.02 图 18

图 18 | 联合观察损失、策略熵、平均梯度范数和回答长度，检查是否出现不稳定或过度集中；训练损失下降不保证任务准确率同步上升。

38.03 数据清理

数据清理 | 删除 Q9，因为舍入误差导致环境把逻辑正确解也判错。此例说明验证器质量会直接影响反馈、教师判断与学习评估。

	Accuracy	Avg accuracy
GRPO	41.2 ± 0.8	38.2 ± 0.0
+ only high-entropy tokens (Wang et al., 2025)	37.8 ± 2.2	35.9 ± 0.1
GSPO (Zheng et al., 2025a)	40.1 ± 2.3	37.7 ± 0.1
CISPO (Chen et al., 2025a)	41.2 ± 1.8	37.8 ± 0.1
SDPO	$\mathbf{48.8} \pm 0.6$	$\mathbf{43.8} \pm 0.0$

39.01 – Table 9: Performance on LCBv6 at/until training step 80 with std over 3 seeds. We compare to GSPO (Zheng et al., 2025a) and CISPO (Chen et al., 2025a). With Qwen3-8B.

39.01 表 9

表 9 | 相同步数比较中，SDPO 代码准确率和训练期平均准确率高于所列基线；此表没有跨任务遗忘指标，不能用它判断能力保留。

Question	**SDPO**	**Best-of-k**	**Multi-turn**	**Speedup** Best-of-k $\rightarrow$ SDPO
1	104	98	**59**	0.9×
3*	**1987**	$\geq$ 2750	$\geq$ 2750	1.4×
10*	**938**	$>$ 2750	1706	2.9×
43	111	**109**	111	1.0×
46*	1852	1466	**1315**	0.8×
59	172	123	**76**	0.7×
69	280	**134**	**134**	0.5×
74*	1948	**1466**	2405	0.8×
86	**85**	421	335	5.0×
91*	**1360**	$\geq$ 2750	2384	2.0×
92*	**1575**	$\geq$ 2750	2203	1.8×
95*	1948	**1466**	1794	0.8×
100	**277**	294	1596	1.1×
103*	2246	$\geq$ 2750	**2210**	1.2×
111	85	95	**39**	1.1×
120	**24**	327	70	13.6×
125*	1795	**1466**	2320	0.8×
127	**28**	368	61	13.1×
129	168	173	**104**	1.0×
Hard tasks	**894**	1145	1141	1.3×
Very hard tasks	**1739**	2180	2121	1.2×

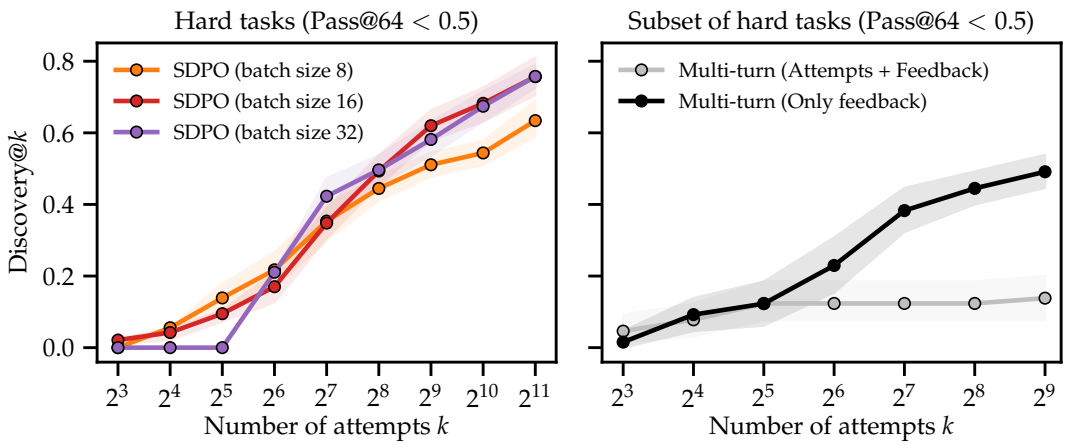
39.02 – Table 10: Mean number of generations until first success per question for SDPO, best-of-k sampling, and the multi-turn sampling. For the mean calculation, values are truncated at the maximum budget of 2750 generations. Very hard tasks (pass@64 < 0.03) are marked with an asterisk (*). Averaged over all questions, SDPO achieves successes faster than the baselines, reaching a speedup of up to 13.6× on individual questions compared to best-of-k sampling.

39.02 表 10

表 10 | 首次成功次数跨题平均前，在 2750 次处截断；困难题/极难题平均加速约 1.3/1.2 倍。它与主文固定发现概率处“3 倍”是不同统计量。

Question	Initial Teacher Accuracy (%)
1	0.00
3	0.00
10	0.00
43	6.25
46	0.00
59	0.00
69	3.12
74	0.00
86	0.00
91	0.00
92	0.00
95	0.00
100	0.00
103	0.00
111	0.00
120	0.00
125	0.00
127	1.23
129	0.06

40.01 – Table 11: **Average accuracy of the retrospective teacher at the first step for each question.** These scores represent the percentage of successful solutions generated when the base model is reprompted with feedback in a single-turn interaction. For the majority of these hard and very hard tasks, the teacher accuracy is near or exactly 0%. Despite this, the self-distilled token-level advantages are sufficiently rich for SDPO to iteratively refine its policy and solve these questions over successive updates.



40.02 – Figure 19: **Ablations self-distillation at test-time on hard tasks. Left:** Impact of SDPO batch size on pass@k curves. While smaller batch sizes (8 and 16) can lead to slightly earlier discoveries at very low generation budgets ($k < 2^6$), larger batch sizes (16, 32) result in more stable updates that significantly improve the discovery rate as the budget scales. **Right:** Comparison of multi-turn reprompting templates on a subset of hard questions. The "Only feedback" template concatenates the feedback from previous attempts using a first-in, first-out sliding window. The "Attempts + Feedback" template concatenates the full turn, also using a sliding window. Including only the feedback substantially outperforms concatenating full conversations.

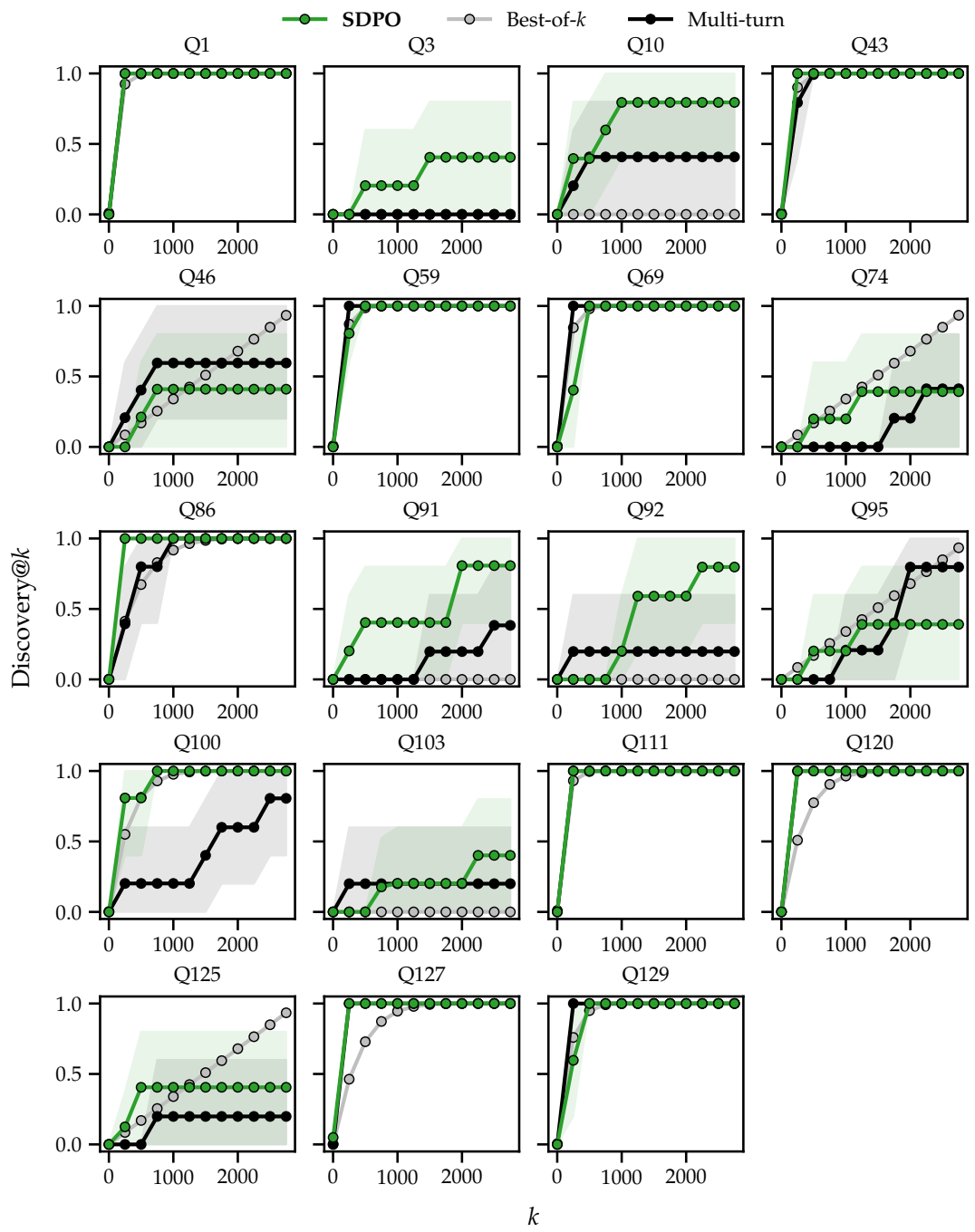
40.01 表 11

表 11 | 初始反馈教师在多数难题上的生成准确率为 0 或接近 0；这不排除分布中某些局部概率偏好仍能指导改进。

40.02 图 19

图 19 | 较小批量可能更早发现解，较大批量后期更稳定；多轮基线仅拼接反馈优于连原尝试一起保留，提示重复错误答案会干扰后续探索。

蓝色数字对应段落 / 图表；术语在首次出现处批注



41.01 — Figure 20: **Individual task results self-distillation at test-time.** Discovery@k for each of the 19 questions evaluated in Section 5. In most cases, SDPO finds a successful solution significantly earlier than both the base model and the multi-turn baseline. Notably, for one question (Q3) where the base model and the multi-turn baseline maintain a discovery@k of zero for the entire budget up to 2750 , SDPO discovers a solution after 321 attempts. Curves represent the mean and 90% confidence intervals across 5 random seeds per question.

41.01 逐题结果

逐题结果｜发现曲线反映题间差异；并非每题 SDPO 都最快。Q3 个案显示它能在基线未成功的预算区间找到解，均值及区间来自每题 5 个种子。

E Experiment Details

E.1 Technical setup

42.01 – All experiments were conducted on a single node equipped with four NVIDIA GH200 GPUs, for a total of 378GB VRAM. Our environment is built on top of the NVIDIA PyTorch container `nvcr.io/nvidia/pytorch:25.02-py3`, with CUDA 12.8 and PyTorch v2.7.0.

42.02 – Our implementation is based on the `verl` library (Sheng et al., 2025). We use PyTorch Fully Sharded Data Parallel (FSDP2) for distributed training. For rollout generation, we employ `vLLM` (Kwon et al., 2023), which enables efficient batched inference on the multi-GPU node.

E.2 Hyperparameters

42.03 – We summarize hyperparameters used for SDPO in Table 12 and those used for GRPO in Table 13.

Parameters	**Without Feedback** Section 3	**With Feedback** Section 4	**TTT** Section 5
General			
Model	Qwen/Qwen3-8B allenai/Olmo3-7B-Instruct	Qwen/Qwen3-8B	Qwen/Qwen3-8B
Thinking	False	False	False
Data			
Max. prompt length	2048	2048	2048
Max. response length	8192	8192	8192
Batching			
Question batch size	32	32	1
Mini batch size	32	1	1
Number of rollouts	8	8	16
Rollout			
Inference engine	vllm	vllm	vllm
Temperature	1.0	1.0	1.0
Validation			
Number of rollouts	16	4	-
Temperature	0.6	0.6	-
Top-p	0.95	0.95	-
SDPO loss			
Top-K distillation	100	20	20
Distillation divergence	Jensen–Shannon	Reverse-KL	Reverse-KL
Clip advantages	–	–	5.0
Teacher-EMA update rate	0.05	0.01	0.01
Rollout importance sampling clip	2	2	2
Training			
Optimizer	AdamW	AdamW	AdamW
Learning rate	1×10^{-5} (constant)	1×10^{-6} (constant)	1×10^{-6} (constant)
Warmup steps	10	0	0
Weight decay	0.01	0.01	0.01
Gradient Clip Norm	1.0	1.0	1.0

Table 12: Hyperparameters used for **SDPO** for each experimental setup.

42.01 硬件与框架

硬件与框架 | 单节点 4 张 GH200，总显存 378 GB；verl、FSDP2 训练和 vLLM 推理。复现时间效率时需记录相同硬件及执行环境。

42.02 实现

实现 | 分布式参数分片降低单卡存储，批量推理提高 rollout 吞吐。推理与训练引擎可能有概率差异，需结合重要性采样校正。

42.03 表 12 重点

表 12 重点 | 无反馈任务使用 JS、top-K=100、EMA=0.05；代码与 TTT 使用 reverse-KL、K=20、EMA=0.01。主文稳定性概述不代表所有实验一律采用 JS。

Parameters	**Experiment 1** Section 3
General	
Model	Qwen/Qwen3-8B allenai/Olmo3-7B-Instruct
Thinking	False
Data	
Max. prompt length	2048
Max. response length	8192
Batching	
Question batch size	32
Mini batch size	8 (default) / 32 (on-policy)
Number of rollouts	8
Rollout	
Inference engine	vllm
Temperature	1.0
Validation	
Temperature	0.6
Top-p	0.95
Number of rollouts	16
Loss	
ϵ-high	0.28
Rollout importance sampling clip	2
KL coefficient (λ)	0.0
Training	
Optimizer	AdamW
Learning rate	1×10^{-6} (default) / 1×10^{-5} (on-policy)
Warmup steps	10
Weight decay	0.01
Gradient Clip Norm	1.0

43.02 –

Table 13: Hyperparameters used for **GRPO**.

E.2.1 Details on Hyperparameter Selection (Section 3)

For GRPO in the experiments in Section 3, we perform a grid search over learning rates $\{10^{-5}, 10^{-6}\}$ and minibatch sizes $\{8, 32\}$. For on-policy GRPO, we search over the same learning rates while fixing the minibatch size to 32. For SDPO, we grid-search over KL variants (forward KL, Jensen–Shannon), learning rates $\{10^{-5}, 10^{-6}\}$, and minibatch sizes $\{8, 32\}$. For each method (GRPO, on-policy GRPO, and SDPO), we select a *single* hyperparameter configuration that achieves the highest validation accuracy within the first 5 hours of training, evaluated across all datasets and models used in Section 3. We further report results obtained by selecting the optimal hyperparameter configuration separately for each model and dataset in Table 3.

43.01 表 13

表 13 | GRPO 的默认/在策略版本具有不同小批量与学习率；KL 系数为零。复现时不能只替换算法名称而忽略实际更新次数与超参数。

43.02 调参协议

调参协议 | 在学习率、批量及散度变体中网格搜索，选前 5 小时验证准确率最高配置。正文最后一句的表号与表 7 说明不一致，应结合表注确认单任务调参结果。

E.3 User Templates

44.01 – For multiple-choice questions and tool use, the model must be prompted in a task-specific manner. We therefore provide the prompt templates used for these settings below.

```
Given a question and four options, please select the right answer. Respond in the
    following format:
<reasoning>
...
</reasoning>
<answer>
...
</answer>

For the answer, only output the letter corresponding to the correct option (A, B, C,
     or D), and nothing else. Do not restate the answer text. For example, if the
    answer is "A", just output:
<answer>
A
</answer>
```

Listing 1: **System prompt: Multiple Choice Questions**

```
{question}
Please reason step by step.
```

Listing 2: **User prompt: Multiple Choice Questions**

44.02 –

```
Your task is to answer the user's question using available tools.
You have access to the following tools:
Name: Axolotl
Description: Collection of axolotl pictures and facts
Documentation:
getRandomAxolotlImage: Retrieve a random axolotl image with information on the
    image source.
Parameters: {}
Output: Successful response.
 - Format: application/json
 - Structure: Object{url, source, description}
searchAxolotlImages: Search for axolotl images based on specific criteria such as
    color, gender, and size.
Parameters: {"color": "string. One of: [wild, leucistic, albino]. The color of the
    axolotl (e.g., 'wild', 'leucistic', 'albino', etc.).", "gender": "string. One
    of: [male, female]. The gender of the axolotl ('male', 'female').", "size": "
    string. One of: [small, medium, large]. The size of the axolotl ('small', '
    medium', 'large').", "page": "integer. The page number for pagination purposes
    ."}
Output: Successful response.
 - Format: application/json
 - Structure: Object{results: Array[Object{url, source, description}], pagination:
    Object{current_page, total_pages, total_results}}
getAxolotlFacts: Retrieve interesting facts about axolotls such as their habits,
    habitats, and physical characteristics.
Parameters: {"category": "string. One of: [habits, habitat, physical
    characteristics]. The category of facts to retrieve (e.g., 'habits', 'habitat',
     'physical characteristics').", "limit": "integer. The maximum number of facts
    to return."}
Output: Successful response.
 - Format: application/json
 - Structure: Array[Object{fact, source}]
```


44.01 模板目的

模板目的 | 多选题固定 reasoning/answer 标签，答案仅输出选项字母；使验证器可可靠解析。提示属于论文实验材料，不是本次文件处理的指令。

44.02 工具模板

工具模板 | 列出工具名、功能、输入 JSON 类型与返回结构，任务是将请求映射到正确调用。格式错误也可能导致奖励失败，应与知识推理错误区分。

```
Use the following format:
Thought: you should always think about what to do
Action: the action to take, should be one of the tool names.
Action Input: the input to the action, must be in JSON format. All of the action
    input must be realistic and from the user.

Begin!
Question: Hey, can you show me a random picture of an axolotl?
```

Listing 3: **Example user prompt: Tool use**

45.01 工具调用格式

工具调用格式 | Thought、Action 与 JSON Action Input 分开，便于环境执行并返回反馈。本页是提示示例续篇，不需要把每个参数词重复解释。

F Qualitative Examples

F.1 Visualization of Advantages

46.01 – Figure 21 compares the advantages of SDPO and GRPO in a representative example.

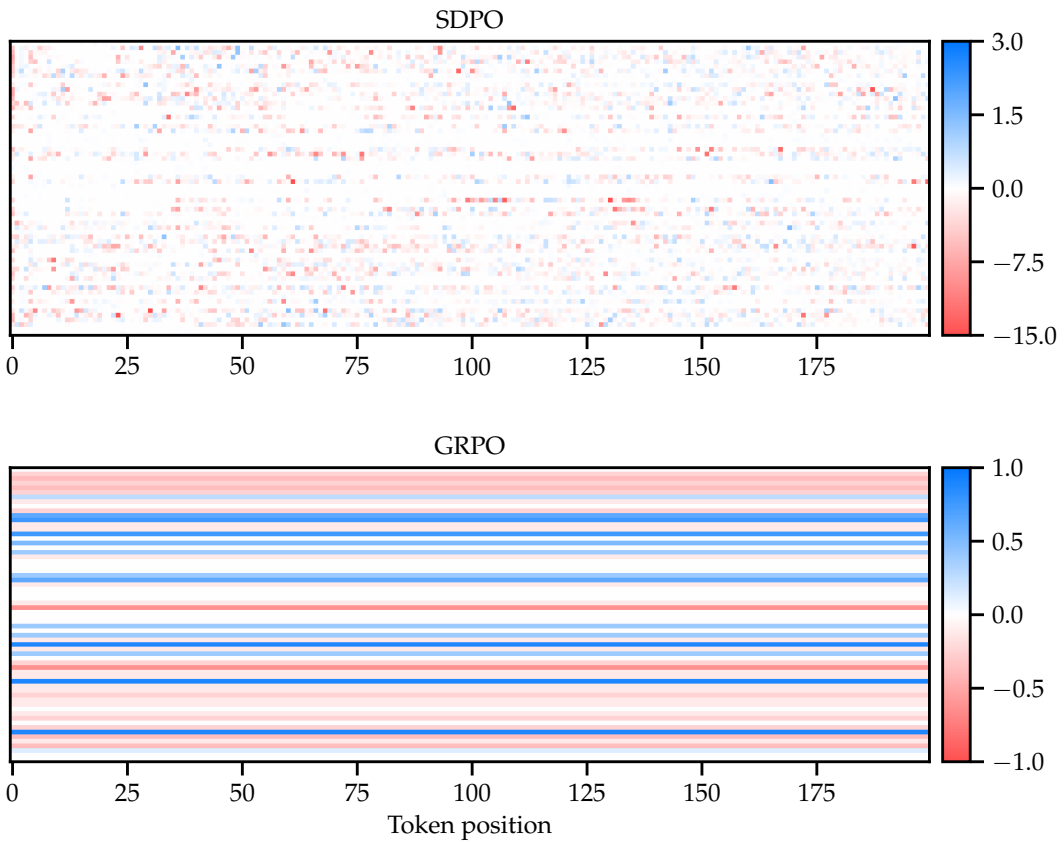


Figure 21: Visualization of advantages in SDPO and GRPO with Olmo3-7B-Instruct in a batch from the Chemistry task of Section 3. Each row corresponds to the beginning of a response. The color indicates the advantage value at that token position, with positive advantages shown in blue and negative advantages shown in red.

F.2 Examples

46.02 – Below, we show an example from training SDPO on LCBv6 using Qwen3-8B.

```
[Prompt]

You are a coding expert. You will be given a coding problem, and you need to write a
     correct Python program that matches the specification and passes all tests. The time
     limit is 1 second. You may start by outlining your thought process. In the end,
     please provide the complete code in a code block enclosed with ``` ```.

You are given a binary string s of length n, where:

'1' represents an active section.
'0' represents an inactive section.

You can perform at most one trade to maximize the number of active sections in s. In a
     trade, you:
```


46.01 图 21

图 21 | SDPO 优势在位置间变化，局部正向或负向；GRPO 同一响应中接近常量。密集分配不意味着每个位置都产生大梯度，很多位置教师与学生一致。

46.02 案例入口

案例入口 | 后续展示原题、学生生成和运行反馈。代码含错误，目的是说明反馈链路，不应把模型生成内容当作已验证正确实现。

47.01 —

```
Convert a contiguous block of '1's that is surrounded by '0's to all '0's.
Afterward, convert a contiguous block of '0's that is surrounded by '1's to all '1's.

Return the maximum number of active sections in s after making the optimal trade.
Note: Treat s as if it is augmented with a '1' at both ends, forming t = '1' + s + '1'.
     The augmented '1's do not contribute to the final count.
```

47.02 —

```
Example 1:

Input: s = "01"
Output: 1
Explanation:
Because there is no block of '1's surrounded by '0's, no valid trade is possible. The
     maximum number of active sections is 1.

Example 2:

Input: s = "0100"
Output: 4
Explanation:

String "0100" -> Augmented to "101001".
Choose "0100", convert "101001" -> "100001" -> "111111".
The final string without augmentation is "1111". The maximum number of active sections is
      4.


Example 3:

Input: s = "1000100"
Output: 7
Explanation:

String "1000100" -> Augmented to "110001001".
Choose "000100", convert "110001001" -> "110000001" -> "111111111".
The final string without augmentation is "1111111". The maximum number of active sections
      is 7.


Example 4:

Input: s = "01010"
Output: 4
Explanation:

String "01010" -> Augmented to "1010101".
Choose "010", convert "1010101" -> "1000101" -> "1111101".
The final string without augmentation is "11110". The maximum number of active sections
     is 4.
```

47.03 —

```
Constraints:

1 <= n == s.length <= 10^5
s[i] is either '0' or '1'

Your solution should have the following signature: ```python
def maxActiveSectionsAfterTrade(s: str) -> int:
```


[Response]

<think>

</think>
```

47.04 —

```
To solve this problem, we need to understand the structure of the string and the trade
     operation. Here's the plan:

### Key Insights:
1. We are given a binary string `s`, and we are allowed **at most one trade**.
2. A **trade** consists of two steps:
   - Convert a contiguous block of `'1'` that is **surrounded by `'0'` into `'0'` (i.e.,
     flip it).
   - Then convert a contiguous block of `'0'` that is **surrounded by `'1'` into `'1'` (i
     .e., flip it).
3. The actual count of active sections is the number of `'1'` characters **after** the
     trade. The augmented `'1'` at the beginning and end do not count in the final result.
```

47.01 题意

题意 | 先将被 0 包围的 1 块变为 0，再把被 1 包围的 0 块变为 1，最多操作一次；两端虚拟 1 不计入答案。边界条件是潜在出错点。

47.02 样例

样例 | 从无可行操作到可把相邻 0 块连通，样例约束最优 trade 的语义；不能随意把处在边界的 1 块也当作可移除块。

47.03 规模约束

规模约束 | n 可达 10^5，需要线性或接近线性的算法。后文枚举与模拟可能超时，不能只通过小样例判断解法可用。

47.04 学生分析

学生分析 | 复述两步操作和虚拟边界，计划寻找连续区块；此处只是模型自己的思路陈述，还须检查后续代码是否真正遵守规则。
```

48.01 —

```
### Strategy:
- We simulate the trade by considering each possible way to flip a block of `'1'` (
    bounded by `'0'`) and then a block of `'0'` (bounded by `'1'`).
- For each possible valid block, simulate the trade and compute the resulting number of
    `'1'` characters.
- We only need to consider the original string `s`, not the augmented one (since we can
    simulate the trade directly on `s` by checking boundaries).

We'll use a sliding window technique to find contiguous blocks of `'1'` and `'0'` and
    simulate the trade.

---
```

48.02 —

```
### Implementation Steps:
1. Simulate the trade by checking for a block of `'1'` surrounded by `'0'` or at the
    edges (we simulate the augmented `1` at both sides).
2. For each such block, simulate converting it to `'0'`, then convert a `'0'` block (
    again, surrounded by `'1'`) to `'1'`.
3. Count the number of `'1'` in the final string and track the maximum.

---

### Python Code:
```python
```

48.03 —

```
def maxActiveSectionsAfterTrade(s: str) -> int:
    if not s:
        return 0

    max_ones = 0
    n = len(s)

    # We are simulating the augmented '1's on both ends, so we treat the boundaries as
     '1's
    # So '0' at index 0 or n-1 is considered surrounded by '1's

    # Step 1: Find all candidate positions for the first part of the trade
    # (convert a block of 1s surrounded by 0s into 0s)
    for i in range(n):
        if s[i] == '1':
            start = i
            while i < n and s[i] == '1':
                i += 1
            end = i - 1
            # Check if this block is surrounded by '0's
            left_boundary = (start == 0 or s[start - 1] == '0')
            right_boundary = (end == n - 1 or s[end + 1] == '0')
            if left_boundary and right_boundary:
                # This is a candidate block of '1's to flip
                # Now, look for a block of '0's surrounded by '1's in the remaining
    string
                # Replace the '1' block with '0's
                # Now find a '0' block surrounded by '1's
                # We can simulate the remaining string
                temp = list(s)
                for j in range(start, end + 1):
                    temp[j] = '0'

                # Now find a contiguous block of '0's surrounded by '1's
                max_trade_ones = 0
                for k in range(n):
                    if temp[k] == '0':
                        start0 = k
                        while k < n and temp[k] == '0':
                            k += 1
                        end0 = k - 1
                        # Check if this block is surrounded by '1's
                        left_0 = (start0 == 0 or temp[start0 - 1] == '1')
                        right_0 = (end0 == n - 1 or temp[end0 + 1] == '1')
                        if left_0 and right_0:
                            # Flip this block of '0's to '1's
                            for j in range(start0, end0 + 1):
                                temp[j] = '1'
                            # Count the number of '1's
                            count = sum(temp)
                            max_ones = max(max_ones, count)
                            break  # Only take the first valid block for this trade
                # After trying this '1' block, restore the original string
                # No need, we just used a copy
    return max_ones
```
```

48.01 策略

策略 | 学生计划枚举区块并模拟两次翻转；较多重复扫描，且“只用原串模拟边界”的处理容易与题目虚拟边界语义不一致。

48.02 实现计划

实现计划 | 分别找候选 1 块、转换后 0 块、统计最终 1 数；这些步骤并未给出整体复杂度保证。

48.03 代码检查

代码检查 | for 循环中修改 i/k 不会改变下一轮迭代来源；sum(temp) 对字符串列表求和也有类型错误。保留论文原始错误，观察环境如何定位它。

```
---

### Optimization Note:
The above approach is **not optimal** for very large strings. A more efficient method
     would involve precomputing the positions of blocks of `'1'` and `'0'`, and checking
     valid trades between these blocks in linear time.

But with the constraints given (up to 10^5), we need to ensure linear or near-linear
     performance. The current code is correct but **not efficient** for large inputs.

Would you like me to provide a more optimized version that runs in **O(n)** time and uses
      only one pass through the string?


[Feedback]

b'Runtime Error\nTypeError: unsupported operand type(s) for +: \'int\' and \'str\'\nLine
     48 in maxActiveSectionsAfterTrade (Solution.py)\n\nLast Executed Input\n"11000"'
```

F.3 Environment Feedback

We show three examples of feedback in our coding environment, inspired by LeetCode.

```
Test Case 3: Wrong Answer

Input
[[3,6,1],[4,21,4]]

Output
6

Expected
22.87500

Test Case 6: Wrong Answer

Input
[[12,25,3],[3,14,2]]

Output
14

Expected
25.83333
```

Listing 4: Example of feedback "Wrong Answer" from our code environment in case of a wrong answer, inspired by LeetCode

```
Runtime Error
MemoryError:
Line 91 in <module> (Solution.py)
Line 25 in solve (Solution.py)

Last Executed Input
10
633 9312
1314 8548
8857 1062
6410 3289
8594 1263
8549 733
3858 5973
... (3 more lines)
```


49.01 自我评价

自我评价 | 学生声称“代码正确但低效”，实际运行却报错。说明模型自然语言自评不可靠，需要环境反馈核验。

49.02 具体反馈

具体反馈 | TypeError 指向 sum(temp)：temp 元素是字符，整数累加不能直接加字符串。行号和失败输入可帮助教师局部定位，不只是告诉模型答案错了。

49.03 Wrong Answer

Wrong Answer | 列出失败输入、实际输出和期望输出，提供数值反例。教师需要结合问题理解反例，反馈本身不是现成的逐词元奖励。

49.04 MemoryError

MemoryError | 暴露内存资源约束及触发位置，说明丰富反馈可描述性能失败，而不只描述答案内容错误。

Listing 5: Example of feedback “Memory Error” from our code environment in case of a wrong answer, inspired by LeetCode

```
Runtime Error
IndexError: list index out of range
Line 28 in sortMatrix (Solution.py)

Last Executed Input
[[-1,-1,-1,-1,-1,-1,-1,-1,...
```

Listing 6: Example of feedback “Index Error” from our code environment in case of a wrong answer, inspired by LeetCode

F.4 Illustrative Example

Figure 22 shows an illustrative example of the dense credit assignment in SDPO.

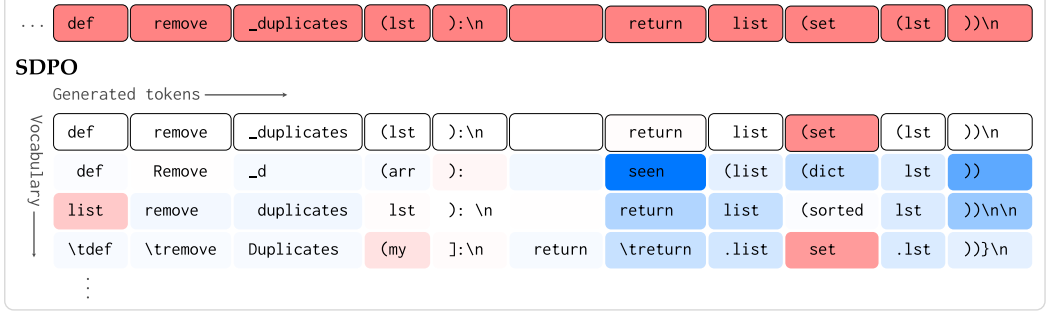


Figure 22: **Dense credit assignment through self-teaching in SDPO.** The answer is generated by then model (Qwen3-8B) before seeing the feedback. Then, we re-evaluate the log-probs of the original attempt with the self-teacher after seeing the feedback. We show the per-token $\log(\mathbb{P}^{(\text{self-teacher})}/\mathbb{P}^{(\text{student})})$, with red indicating negative values (self-teacher disagrees), blue indicating positive values (teacher reinforces), and white indicating values around zero. Using binary rewards, GRPO would assign the same, negative advantage to all tokens in the sequence. In contrast, SDPO turns the feedback into dense credit assignment across the sequence. The first row shows the tokens of the generated response. The 3 other rows show the top-k logits of the self-teacher that are used during self-distillation, suggesting alternative tokens. Notably, in this example, the self-teacher identifies the error through retrospection without an explicit solution. The credit assignment on the generated sequence, and the alternative top-k logits correctly show that replacing `set` with `dict` maintains the order of elements. Further, in the seventh shown position, the model also identifies an alternative solution path which starts with the `seen` token, instead of directly returning the output. The activation is sparse, identifying where mistakes happen and adjusting to the students' response distribution for specifically these few tokens.

50.01 IndexError

IndexError | 越界异常、行号及最后输入提供局部诊断线索。它能提示索引/边界问题，但未必给出完整正确算法。

50.02 图 22

图 22 | 原回答使用 set，教师倾向 dict 以保留顺序，并提示另一路径 seen。局部概率比显示它在哪里否定原词元、在哪里鼓励替代词元。

50.03 图意总结

图意总结 | GRPO 会给全响应相同负优势；SDPO 根据反馈区分局部错误和可保留内容。定性热图说明机制，整体收益仍以受控实验为依据。